IEEE Std 802.1ASed™-2026
(Amendment to IEEE Std 802.1AS™-2025)

IEEE Standard for Local and Metropolitan Area Networks—

Timing and Synchronization for Time-Sensitive Applications

Amendment 1: Fault-Tolerant Timing with Time Integrity

Developed by the

LAN/MAN Standards Committee
of the
IEEE Computer Society

Approved xx February 2026

IEEE SA Standards Board

Abstract: This amendment to IEEE Std 802.1AS™-2025 specifies protocols, processes, procedures, functions, mechanisms, and managed objects to enable fault-tolerant timing by increasing the availability of the time and adding time integrity. This is achieved using two or more generalized precision time protocol (gPTP) domains, multiple time distribution paths, the local oscillator clock, and a time selection function with individual processes for times that have interdependencies and times that do not have interdependencies. Fault-tolerant timing includes fault-tolerant time generation and distribution.

Keywords: amendment, dependent gPTP times, fault-tolerant timing, IEEE 802.1AS™, IEEE 802.1ASed™, independent gPTP times, time selection function

The Institute of Electrical and Electronics Engineers, Inc.
3 Park Avenue, New York, NY 10016-5997, USA

PDF: ISBN 978-0-7381-xxxx-x STDxxxxx
Print: ISBN 978-0-7381-xxxx-x STDPDxxxxx

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (https://standards.ieee.org/ipr/disclaimers.html), appear in all IEEE standards and may be found under the heading "Important Notices and Disclaimers Concerning IEEE Standards Documents."

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers involved in technical working groups are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning all standards, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not guarantee safety, security, health, or environmental protection, or compliance with law, or guarantee against interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE Standards documents are supplied "AS IS" and "WITH ALL FAULTS."

Use of an IEEE standard is wholly voluntary. The existence of an IEEE standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document should rely upon their own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Translations

The IEEE consensus balloting process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English language version published by IEEE is the approved IEEE standard.

Use by artificial intelligence systems

In no event shall material in any IEEE Standards documents be used for the purpose of creating, training, enhancing, developing, maintaining, or contributing to any artificial intelligence systems without the express, written consent of the IEEE SA in advance. "Artificial intelligence" refers to any software, application, or other system that uses artificial intelligence, machine learning, or similar technologies, to analyze, train, process, or generate content. Requests for consent can be submitted using the Contact Us form.[1]

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual is not, and shall not be considered or inferred to be, the official position of IEEE or any of its committees and shall not be considered to be or be relied upon as, a formal position of IEEE or IEEE SA. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter's views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group. Statements made by volunteers may not represent the formal position of their employer(s) or affiliation(s). News releases about IEEE standards issued by entities other than IEEE SA should be considered the view of the entity issuing the release rather than the formal position of IEEE or IEEE SA.

Comments on standards

Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents**.

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of Governors are not able to provide an instant response to comments or questions except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or revisions to an IEEE standard is welcome to join the relevant IEEE SA working group. You can indicate interest in a working group using the Interests tab in the Manage Profile & Interests area of the IEEE SA myProject system.[2] An IEEE Account is needed to access the application.

Comments on standards should be submitted using the Contact Us form.[1]

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory

[1] Available at: https://standards.ieee.org/about/contact/.

[2] Available at: https://development.standards.ieee.org/myproject-web/public/view.html#landing.

requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

Copyrights

IEEE draft and approved standards are copyrighted by IEEE under U.S. and international copyright laws. They are made available by IEEE and are adopted for a wide variety of both public and private uses. These include both use, by reference, in laws and regulations, and use in private self-regulation, standardization, and the promotion of engineering practices and methods. By making these documents available for use and adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright to the documents.

Photocopies

Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license to photocopy portions of any individual standard for company or organizational internal use or individual, non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400; https://www.copyright.com/. Permission to photocopy portions of any individual standard for educational classroom use can also be obtained through the Copyright Clearance Center.

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit IEEE Xplore or contact IEEE.[3] For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata

Errata, if any, for all IEEE standards can be accessed on the IEEE SA Website.[4] Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the

[3] Available at: https://ieeexplore.ieee.org/browse/standards/collection/ieee.

[4] Available at: https://standards.ieee.org/standard/index.html.

Additional Resources Details section. Errata are also available in IEEE Xplore. Users are encouraged to periodically check for errata.

Patents

IEEE standards are developed in compliance with the IEEE SA Patent Policy.[5]

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE SA Website at https://standards.ieee.org/about/sasb/patcom/patents.html. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

IMPORTANT NOTICE

Technologies, application of technologies, and recommended procedures in various industries evolve over time. The IEEE standards development process allows participants to review developments in industries, technologies, and practices, and to determine what, if any, updates should be made to the IEEE standard. During this evolution, the technologies and recommendations in IEEE standards may be implemented in ways not foreseen during the standard's development. IEEE standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, data privacy, and interference protection practices and all applicable laws and regulations.

[5] Available at: https://standards.ieee.org/about/sasb/patcom/materials.html.

Participants

At the time this standard was submitted to the IEEE SA Standards Board for approval, the IEEE 802.1 Working Group had the following membership:

Glenn Parsons, *Chair*
Jessy Rouyer, *Vice Chair*
János Farkas, *TSN Task Group Chair*
Silvana Rodrigues, *Editor IEEE Std 802.1AS*
Abdul Jabbar, *Editor P802.1ASed*

Mahin Ahmed
Venkat Arunarthi
Ralf Assmann
Christian Boiger
Paul Bottorff
Radhakrishna Canchi
Feng Chen
Lihao Chen
Abhijit Choudhury
Donald Fedyk
Geoffrey Garner
Fumihide Goto
Stephen Haddock
Mark Hantel
Marc Holness
Daniel Hopf
Woojung Huh
Yoshihiro Ito
Michael Karl
Stephan Kehrer
Marcel Kiessling
Gavin Lai
Yunping (Lily) Lyu
Christophe Mangin
Scott Mansfield
Olaf Mater
David McCall
Martin Mittelberger
Hiroki Nakano
Takumi Nomura
Richie Pearn
Dieter Proell
Karen Randall
Alon Regev
Maximilian Riegel
Rajeev Roy
Atsushi Sato
Mick Seaman
Maik Seewald
Ramesh Sivakolundu
Johannes Specht
Marius Stanica
Max Turner
Ganesh Venkatesan
Tongtong Wang
Leon Wessels
Ludwig Winkel
Jordon Woods
Takahiro Yamaura
Nader Zein

The following members of the individual balloting committee voted on this standard. Balloters may have voted for approval, disapproval, or abstention.

Mahin Ahmed
Thomas Alexander
Boon Chong Ang
Butch Anton
Greg Armstrong
Douglas Arnold
Murugan Balraj
Denis Beaudoin
Harry Bims
Christian Boiger
Ashley Butterworth
William Byrd
Pin Chang
Chi-Hua Chen
Rodney Cummings
Samer Darras
Sergio Diaz
János Farkas
Donald Fedyk
Avraham Freedman
Devon Gayle
Roman Graf
Stefan Hagen
Mark Hantel
Xiang He
Marco Hernandez
Werner Hoelzl
Oliver Holland
Woojung Huh
James Innis
Abdul Jabbar
Raj Jain
SangKwon Jeong
Pranav Jha
Stanley Johnson
David Johnston
Lokesh Kabra
Ruslan Karmanov
Piotr Karocki
Kishore Kasichainula
Stuart Kerry
Quist-Aphetsi Kester
Marcel Kiessling
Yongbum Kim
Jeff Koftinoff
Gavin Lai
Jingfei Lv
Michael Lynch
Christophe Mangin
Scott Mansfield
William Rogelio Marchand Nino
Ignacio Marin Garcia
Brett McClellan
Jonathon McLendon
Sven Meier
Martin Mittelberger
Rajesh Murthy
Hamid Naseem
Satoshi Obara
Glenn Parsons
Bansi Patel
Arumugam Paventhan
Brian Petry
Clinton Powell
Dieter Proell
Benjamin Quak
Alon Regev
Maximilian Riegel
Silvana Rodrigues
Benjamin Rolfe
Jessy V. Rouyer
Rajeev Roy
Veselin Skendzic
Eugene Stoudenmire
Mitsutoshi Sugawara
Luisma Torres
Richard Tse
Max Turner
John Vergis
Xiaohui Wang
Stephen Webb
Scott Willy
Ludwig Winkel
Janusz Zalewski

When the IEEE SA Standards Board approved this standard on XX Month 2026, it had the following membership:

Lei Wang, *Chair*
Jon W. Rosdahl, *Vice Chair*
David J. Law, *Past Chair*
Alpesh Shah, *Secretary*

Edward Au
Ted Burse
Xiaofeng (Alfred) Chen
Doug Edwards
Nehad El-Sherif
J. Travis Griffith
Deborah Hagar
Guido R. Hiertz
Ronald W. Hotchkiss
Tyler Jaynes
Thomas Koshy
Howard Li
Xiaohui Liu
Kevin W. Lu
Hiroshi Mano
Daleep Mohla
Annette Reilly
Robby Robson
Dan Sabin
F. Keith Waters
Sha Wei
Luyang (Eric) Zhang

Introduction

This introduction is not part of IEEE Std 802.1ASed-2026, IEEE Standard for Local and Metropolitan Area Networks—Timing and Synchronization for Time-Sensitive Applications—Amendment 1: Fault-Tolerant Timing with Time Integrity.

The first edition of IEEE Std 802.1AS was published in 2011. A first corrigendum, IEEE Std 802.1AS-2011/Cor 1-2013, provided technical and editorial corrections. A second corrigendum, IEEE Std 802.1AS-2011/Cor 2-2015 provided additional technical and editorial corrections.

The second edition, IEEE Std 802.1AS-2020, added support for multiple gPTP domains, Common Mean Link Delay Service, external port configuration, and fine timing measurement for IEEE 802.11 transport. Backward compatibility with IEEE Std 802.1AS-2011 was maintained. A corrigendum, IEEE Std 802.1AS-2020/Cor 1-2021, provides technical and editorial corrections.

The third edition, IEEE Std 802.1AS-2025, is a roll-up of IEEE Std 802.1AS-2020 with the corrigendum IEEE Std 802.1AS-2020/Cor 1-2021 and amendments IEEE Std 802.1ASdr-2024, IEEE Std 802.1ASdn-2024, and IEEE Std 802.1ASdm-2024.

This amendment to IEEE Std 802.1AS-2025 specifies protocols, processes, procedures, functions, mechanisms, and managed objects to enable fault-tolerant timing by increasing the availability of the time and adding time integrity. This is achieved using two or more generalized precision time protocol (gPTP) domains, multiple time distribution paths, the local oscillator clock, and a time selection function with individual processes for times that have interdependencies and times that do not have interdependencies. Fault-tolerant timing includes fault-tolerant time generation and distribution.

Contents

List of figures

List of tables

IEEE Standard for
Local and Metropolitan Area Networks—

Timing and Synchronization for Time-Sensitive Applications

Amendment 1: Fault-Tolerant Timing with Time Integrity

(This amendment is based on IEEE Std 802.1AS-2025.)

NOTE—The editing instructions contained in this amendment define how to merge the material contained therein into the existing base standard and its amendments to form the comprehensive standard.

The editing instructions are shown in ***bold italic***. Four editing instructions are used: change, delete, insert, and replace. ***Change*** is used to make corrections in existing text or tables. The editing instruction specifies the location of the change and describes what is being changed by using ~~strikethrough~~ (to remove old material) and underscore (to add new material). ***Delete*** removes existing material. ***Insert*** adds new material without disturbing the existing material. Deletions and insertions may require renumbering. If so, renumbering instructions are given in the editing instruction. ***Replace*** is used to make changes in figures or equations by removing the existing figure or equation and replacing it with a new one. Editing instructions, change markings, and this NOTE will not be carried over into future editions because the changes will be incorporated into the base standard.[6]

[6] Notes in text, tables, and figures are given for information only, and do not contain requirements needed to implement the standard.

4. Acronyms and abbreviations

Insert the following new abbreviations into Clause 4 in alphanumeric order:

DTSF	dependent time selection function
FTT	fault-tolerant timing (with time integrity)
ITSF	independent time selection function
MVTSA	mid-value time selection algorithm
NQ	not qualified
PPS	pulse per second
TAGP	time agreement generation and preservation
TSF	time selection function

5. Conformance

Change the title and content of 5.3 as follows:

5.3 Time-aware system requirements and options

5.3.1 Time-aware system requirements

An implementation of a time-aware system shall support at least one IEEE 1588 precision time protocol (PTP) Instance.

5.3.2 Time-aware system options

An implementation of a time-aware system may support a fault-tolerant timing with time integrity (FTT) entity, as specified in 20.2.

If YANG is supported with a remote management protocol and if an FTT entity (see 20.2) is supported, the YANG data model ieee802-dot1as-ftt in 17.6.3 shall be supported.

7. Time-synchronization model for a packet network

7.2 Architecture of a time-aware network

7.2.1 General

Insert new list item j) after item i) in 7.2.1 as follows:

j) Fault-tolerant timing with time integrity (see Clause 20) that allows comparison and selection of time from multiple PTP Instances receiving time over dependent and independent synchronization trees.

7.3 Examples of time-aware networks

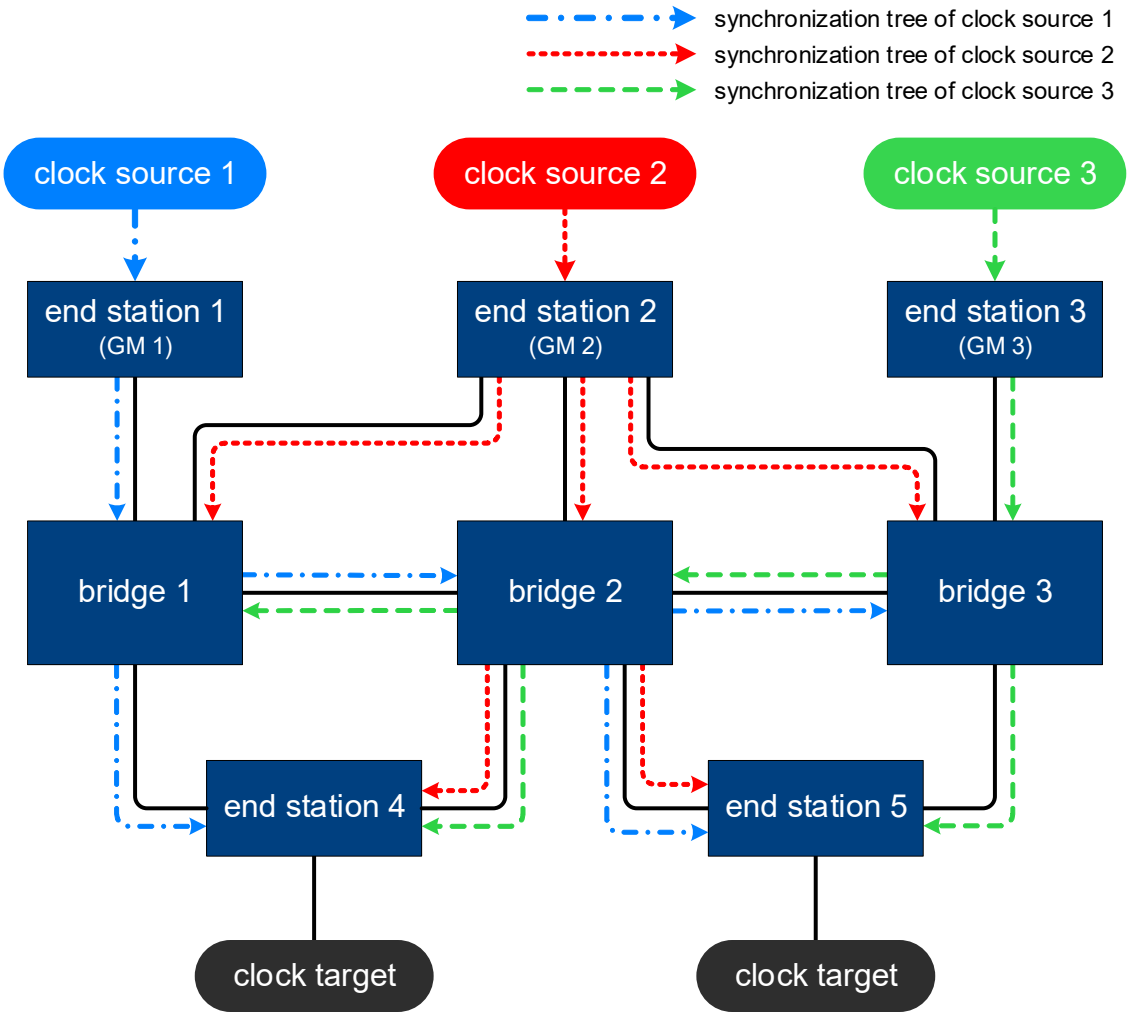
Insert new subclause 7.3.5 after 7.3.4 as follows:

7.3.5 Time-aware network with fault-tolerant timing and time integrity

Figure 7-6a shows an example of a time-aware network with fault-tolerant timing and time integrity (see Clause 20). The network has GM redundancy and path redundancy to all the Bridges and to end stations 4 and 5 to enhance availability and integrity of time in the network.

In this example, three independent clock sources (1, 2, and 3) provide time to three GMs (1, 2, and 3, respectively). These clock sources are synchronized to each other using a time agreement generation and preservation function (see I.4). The synchronization trees distribute the time from each of the three domains (from the three GMs) to all Bridges and to non-GM end stations 4 and 5. FTT entities (see 20.2) in the Bridges and in end stations 4 and 5 select a time from their three inputs. The time selected by each FTT entity has integrity (see 20.1.2.2). The selected time is provided to the local clock targets for use by mechanisms that require synchronized time (e.g., enhancements for scheduled traffic, see 8.6.8.4 in IEEE Std 802.1Q-2022). For example, at end station 4, the times received from GM2 and GM3 share a dependency (see 20.1.2.4) because the synchronization trees from GM2 and GM3 share a common Bridge node (bridge 2). Conversely, the synchronization tree from GM1 to end station 4 does not have any commonalities with the synchronization trees from GM2 and GM3 to end station 4, so the time from GM 1 is independent of the times from GM 2 and GM 3 at end station 4 (see 20.1.2.5). Because of these relationships, the FTT entity of end station 4 first selects between the times from GM 2 and GM 3 using a dependent time selection function (DTSF, see 20.2.3), and then selects between the DTSF instance's selected time and the time from GM 1 using an independent time selection function (ITSF, see 20.2.3) to produce an output time with time integrity. The FTT entity in bridge 2 sees three independent times and, thus, only uses its ITSF to select between them to produce an output time with time integrity.

The network shown in Figure 7-6a can tolerate failure of any one link or device (i.e., maintain the availability of time) as well as detect erroneous time on any one synchronization tree (i.e., maintain the integrity of time). For example, if the link between any two Bridges (bridge 1 to 2 or bridge 2 to 3) fails, all non-GM end stations and Bridges receive time over at least two synchronization trees and select a time using the ITSF. If bridge 1 experiences a fault and sends erroneous time downstream over the GM 1 synchronization tree, all the non-GM end stations and Bridges detect the erroneous time in comparison to the non-faulty time received over the other two synchronization trees. For the example shown in Figure 7-6a, if any one bridge is faulty and introduces errors in the relayed synchronization tree(s), all the other non-faulty bridges and non-GM end stations are able to detect the erroneous times via the FTT entity because the topology allows for at least one non-faulty synchronization tree in the network.



NOTE 1— All the “bridges” in this figure are examples of time-aware systems that contain PTP Relay Instances and an FTT entity. The methods used to terminate time from PTP Relay Instances is not specified in this standard.

NOTE 2—All the end stations in this figure are examples of time-aware systems that contain PTP End Instances and an FTT entity.

NOTE 3—The number of GMs and domains used in an implementation can vary based on the fault tolerance requirements.

Figure 7-6a—Time-aware network example for fault-tolerant timing with time integrity

14. Timing and synchronization management

14.1 General

14.1.1 Data set hierarchy

Change item b) in the lettered list in 14.1.1 as follows:

b) commonServices
   1) commonMeanLinkDelayService
      i) cmldsDefaultDS (see 14.17)
      ii) cmldsLinkPortList[]
         — cmldsLinkPortDS (see 14.18)
         — cmldsLinkPortStatisticsDS (see 14.19)
         — cmldsAsymmetryMeasurementModeDS (see 14.20)
   2) hotStandbyService
      i) hotStandbySystemList[]
         — hotStandbySystemDS (see 14.21)
         — hotStandbySystemDescriptionDS (see 14.22)
   3) fttService
      i) fttEntityList[]
         — fttEntityDS (see 14.23)
         — fttEntityDescriptionDS (see 14.24)

Change the sixth paragraph of 14.1.1 as follows:

The commonServices is an overall structure containing commonMeanLinkDelayService structure for the data sets used for CMLDS, ~~and~~ the hotStandbyService structure for the data sets used for Hot Standby Service, and the fttService structure for the data sets used for fault-tolerant timing with time integrity service.

Insert two new paragraphs at the end of 14.1.1 as follows:

The fttService structure contains the fttEntityList, which is a list of instances of FTT entities. An fftEntityDS and an fttEntityDescriptionDS are maintained for each FTT entity.

Each FTT entity is associated with a set of PTP Instances. The indices of these PTP Instances are mapped to inputs of the FTT entity instance via the member fttPtpInstancetoInput of the fttEntityDS for this instance of the FTT entity.

Insert new subclauses 14.23 and 14.24 after 14.22 as follows:

14.23 Fault-Tolerant Timing with Time Integrity Entity Parameter Data Set (fttEntityDS)

14.23.1 General

The fttEntityDS describes the attributes of the respective instance of the fault-tolerant timing with time integrity entity.

The parameters of fttEntityDS are summarized in Table 14-25 and subsequently specified.

Table 14-25—fttEntityDS

Name	Data type	Operations supported[a]	References
fttMaxInputs	UInteger8	R	14.23.2
fttNumInputs	UInteger8	RW	14.23.3
fttMaxDtsfs	UInteger8	R	14.23.4
fttNumDtsfs	UInteger8	RW	14.23.5
fttDtsfMaxInputs	Uinteger8	R	14.23.6
fttInputsToTsf	Uinteger16[fttNumDtsfs + 1]	RW	14.23.7
fttPtpInstanceToInput	UInteger32[fttNumInputs]	RW	14.23.8
fttHyst	TimeInterval[fttNumInputs] [fttNumInputs]	RW	14.23.9
fttMaxAs	TimeInterval[fttNumInputs] [fttNumInputs]	RW	14.23.10
fttSelChangeThresh	TimeInterval[fttNumDtsfs + 1]	RW	14.23.11
fttRRThresh	UInteger32	RW	14.23.12
fttRRThreshStdDev	UInteger32	RW	14.23.13
fttUseNQ	Boolean[fttNumDtsfs + 1]	RW	14.23.14
fttUseFTTClk	Boolean	RW	14.23.15
fttTsfSelInput	UInteger8[fttNumDtsfs + 1]	R	14.23.16
fttSelInstance	UInteger32	R	14.23.17
fttTsfSelChangeCnt	Uinteger16[fttNumDtsfs + 1]	R	14.23.18
fttInputTrustState	Enumeration2[fttNumInputs]	R	14.23.19
fttTrustState	Enumeration2	R	14.23.20
fttStateChangeCnt	Uinteger16	R	14.23.21

[a] RW = Read/Write access; R = Read only access.

14.23.2 fttMaxInputs

The fttMaxInputs object specifies the maximum number of input ClockTarget interfaces available on the FTT entity. The value ranges from 1 to 16. The default value is implementation specific.

14.23.3 fttNumInputs

The fttNumInputs object specifies the number of enabled inputs to the FTT entity, where an enabled input is one that has been mapped to either a DTSF instance or to the ITSF. The value ranges from 1 to fttMaxInputs.

14.23.4 fttMaxDtsfs

The fttMaxDtsfs object specifies the maximum number of DTSF instances available in the FTT entity. The default value is implementation specific.

14.23.5 fttNumDtsfs

The fttNumDtsfs object specifies the number of DTSF instances currently enabled in the FTT entity, with the value ranging from 0 to fttMaxDtsfs.

14.23.6 fttDtsfMaxInputs

The fttDtsfMaxInputs object gives the maximum number of inputs available on each of the DTSF instances in the FTT entity. The value ranges from 1 to 16. The default value is implementation specific.

NOTE—All DTSFs of one FTT entity share the same limit on the maximum number of inputs allowed.

14.23.7 fttInputsToTsf

The fttInputsToTsf object provides a list of the FTT input index numbers mapped to each time selection function (TSF). fttInputsToTsf[*j*], where *j* is a value from 0 to fttNumDtsfs, provides the bit map of FTT inputs that are assigned to TSF instance *j*. The most-significant bit corresponds to FTT input index 16, the least-significant bit corresponds to FTT input index 1. The number of inputs assigned to any DTSF instance cannot be greater than fttDtsfMaxInputs and the total number of assigned inputs to all TSF instances cannot exceed fttMaxInputs.

14.23.8 fttPtpInstanceToInput

The fttPtpInstanceToInput object provides a mapping between FTT input index number and the PTP instanceIndex number. fttPtpInstanceToInput[*j*] is the instanceIndex number of the PTP Instance that is connected to the FTT input with input index *j*.

14.23.9 fttHyst

This fttHyst object is a two-dimensional array of TimeInterval (see 6.4.3.3) values. The array has a size of fttNumInputs in each dimension. The object fttHyst[*x*][*y*] holds the hysteresis to be added to fttMaxAs[*x*][*y*] (see 14.23.10) for the time values of the two FTT inputs with index numbers *x* and *y*.

14.23.10 fttMaxAs

The fttMaxAs object is a two-dimensional array of TimeInterval (see 6.4.3.3) values. The array has a size of fttNumInputs in each dimension. The fttMaxAs[*x*][*y*] object defines the maximum expected skew between time values provided by the FTT inputs with index *x* and *y*, when those time values are not faulty.

14.23.11 fttSelChangeThresh

The fttSelChangeThresh object provides the TimeInterval thresholds used by each TSF instance to determine the time difference required to switch between FTT inputs that meet the selection criteria (see 20.3.3.4). fttSelChangeThresh[*j*] is the threshold value used by TSF instance with instance number *j*.

14.23.12 fttRRThresh

The fttRRThresh object specifies the maximum rate-ratio offset magnitude, between the FTT entity's selected time clock rate and the FTT entity's local clock (FTT_CLK) rate, that is deemed to be acceptable to go to or remain in the FREQ_TRUSTED state (see 20.3.4). The rateRatio offset magnitude is expressed as a fractional frequency offset multiplied by 2^{41}. The default value for fttRRThresh is 0.

14.23.13 fttRRThreshStdDev

The fttRRThreshStdDev object specifies the maximum standard deviation of the rate-ratio offset, between the FTT entity's selected time clock rate and the FTT entity's local clock (FTT_CLK) rate, that is deemed to be acceptable to go to or remain in the FREQ_TRUSTED state (see 20.3.4). The standard deviation of the rateRatio offset is expressed as a fractional frequency offset multiplied by 2^{41}, and has a default value of 0.

14.23.14 fttUseNQ

The fttUseNQ object enables or disables the use of not qualified (NQ) input (see 20.2.4) by a TSF instance when no trusted time is found. fttUseNQ[*j*], where *j* is a value from 0 to fttNumDtsfs, is the boolean value of fttUseNQ for TSF instance *j*.

If fttUseNQ[*j*] is TRUE, the use of the NQ input of TSF instance *j* is enabled.

If fttUseNQ[*j*] is FALSE, the use of the NQ input of TSF instance *j* is not enabled.

The default values are implementation-specific.

14.23.15 fttUseFTTClk

The fttUseFTTClk object defines whether the FTT_CLK frequency is used as a reference for checking time integrity (see 20.2.4).

If fttUseFTTClk is TRUE, the FTT_CLK frequency is used as a reference for checking time integrity.

If fttUseFTTClk is FALSE, the FTT_CLK frequency is not used as a reference for checking time integrity.

The default value is implementation-specific.

14.23.16 fttTsfSelInput

The fttTsfSelInput object specifies the FTT input index that is selected by each TSF instance. It is the array value of the global variable fttTsfSelInput (see 20.3.2.3).

14.23.17 fttSelInstance

The fttSelInstance object specifies the instanceIndex of the PTP Instance corresponding to the FTT input that was selected by the FTT entity as its output.

14.23.18 fttTsfSelChangeCnt

The fttTsfSelChangeCnt object counts the number of times each TSF has changed its selection, i.e., selected input that is determined to be trusted. fttTsfSelChangeCnt[*j*], where *j* is a value from 0 to fttNumDtsfs, is the number of times TSF instance *j* has changed its selected input. The default value is 0.

14.23.19 fttInputTrustState

The trust state of all FTT inputs, i.e., the array value of the global variable fttInputTrustState (see 20.3.2.2).

14.23.20 fttTrustState

The value of fttTrustState is the trust state of the FTT entity, i.e., the value of the global variable fttTrustState (see 20.3.2.1).

14.23.21 fttStateChangeCnt

The fttStateChangeCnt object counts the number of times the FTT entity has changed its trust state (see 14.23.20). The default value is 0.

14.24 Fault-Tolerant Timing with Time Integrity Entity Description Parameter Data Set (fttEntityDescriptionDS)

14.24.1 General

The fttEntityDescriptionDS contains descriptive information for the respective instance of the fault-tolerant timing with time integrity entity.

14.24.2 userDescription

The user description is a character string whose maximum length is 128.

14.24.3 fttEntityDescriptionDS table

There is one fttEntityDescriptionDS table per FTT entity, as detailed in Table 14-26.

Table 14-26—fttEntityDescriptionDS

Name	Data type	Operations supported[a]	References
userDescription	Octet128	RW	14.24.2

[a] RW = Read/Write access.

17. YANG data model

17.1 YANG framework

17.1.1 Relationship to the IEEE Std 1588 data model

Change the first paragraph in 17.1.1 as follows:

The YANG data models specified in this standard are based on, and augment, those specified in IEEE Std 1588. In particular the ieee802-dot1as-gptp.yang module imports the ieee1588-ptp-tt module as a whole, augmenting that module as necessary to meet the requirements of this standard. ~~In addition, t~~The ieee802-dot1as-hs.yang module imports the ieee1588-ptp-tt and ieee802-dot1as-gptp modules as a whole, augmenting those modules as necessary to meet the requirements of this standard. The ieee802dot1as-ftt.yang module imports the ieee1588-tt and ieee802-dot1as-gptp modules as a whole, augmenting those modules as necessary to meet the requirements of this standard.

Change the fourth paragraph in 17.1.1 as follows:

The YANG modules of this clause (ieee802-dot1as-gptp.yang, ~~and~~ ieee802-dot1as-hs.yang, and ieee802-dot1as-ftt.yang) use the YANG "import" statement to import the YANG module of IEEE Std 1588e. This effectively uses the IEEE Std 1588 YANG tree as the foundation of the IEEE Std 802.1AS YANG tree. By importing the tree and its data set containers, all members from Clause 14 that are derived from IEEE Std 1588 are also imported.

17.2 IEEE 802.1AS YANG data model

Change the seventh paragraph of 17.2 as follows:

Figure 17-4 provides detail for the common services, including each data set member. The CMLDS has a data set for the service itself (default-ds), and data sets for each Link Port. The Hot Standby Service has data sets for each HotStandbySystem. The FTT service has data sets for each FTT entity.

Replace Figure 17-1 with the following figure:

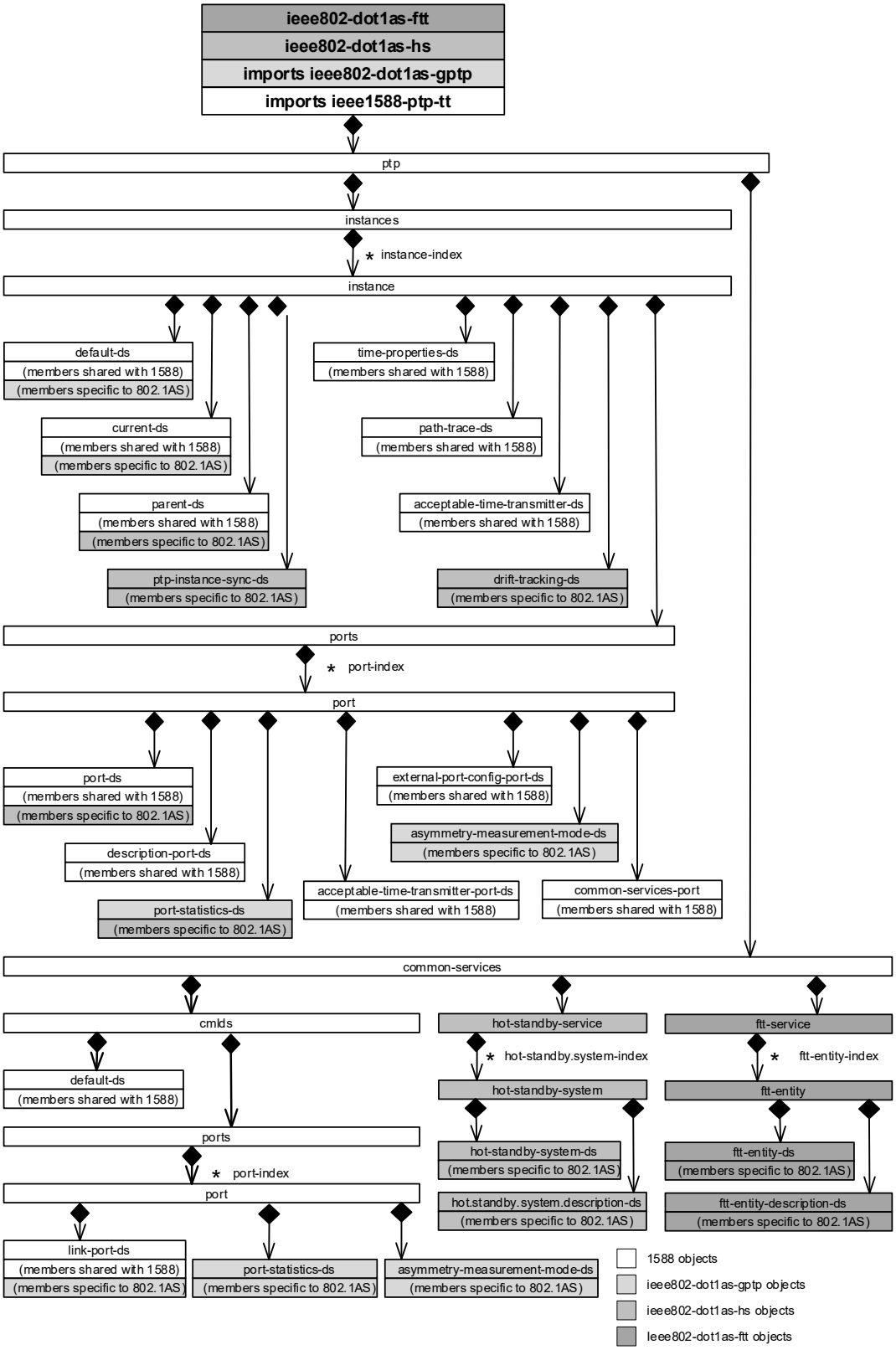


Figure 17-1—Overview of YANG tree

Replace Figure 17-4 with the following figure:

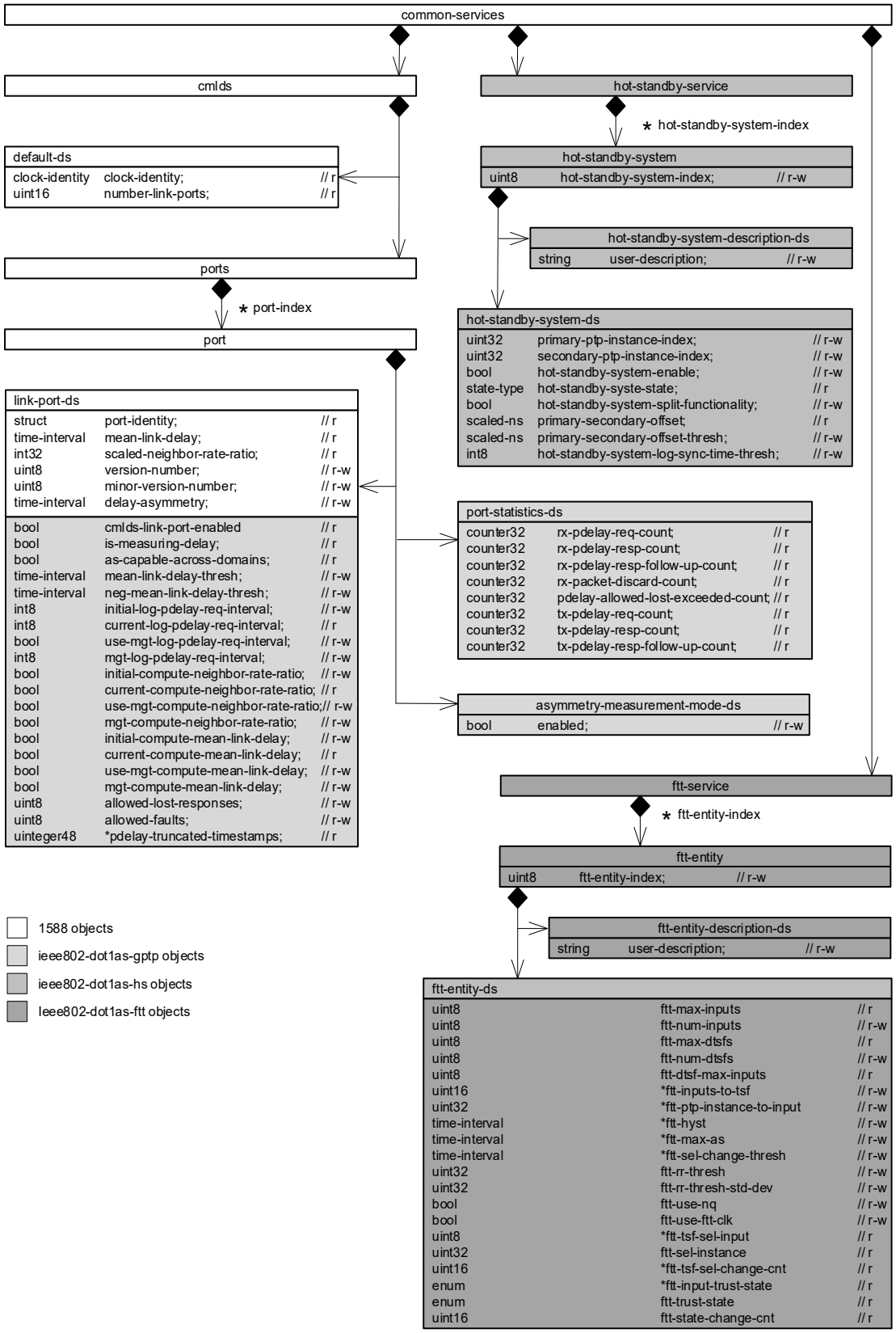


Figure 17-4—Common services detail

17.3 Structure of the YANG data model

Change Table 17-1 as follows:

Table 17-1—Summary of the YANG modules

Module	Managed functionality	YANG specification notes
ietf-yang-types	Type definitions	IETF RFC 6991—Common YANG Data Types.
ieee1588-ptp-tt	Clause 14	IEEE Std 1588e—MIB and YANG Data Models. it imports this YANG module as its foundational tree, including a subset of members from Clause 14.
ieee802-dot1as-gptp	Clause 14	The YANG module of this clause uses YANG augments to add members from Clause 14 that are unique to IEEE Std 802.1AS.
ieee802-dot1as-hs	Clause 14	Hot Standby—YANG Data Model. The YANG module of this clause uses YANG augments to add members from Clause 14 that are unique to hot standby and drift tracking features.
ieee802-dot1as-ftt	Clause 14	IEEE Std 802.1ASed—YANG Data Model. The YANG module of this clause uses YANG augments to add members from Clause 14 that are unique to IEEE Std 802.1ASed.

17.5 YANG schema tree definitions

Insert 17.5.3 at the end of 17.5 as follows:

17.5.3 Tree diagram for ieee802-dot1as-ftt.yang

```
module: ieee802-dot1as-ftt

  augment /ptp-tt:ptp/ptp-tt:common-services:
    +--rw ftt-service {ftt}?
       +--rw ftt-entity* [ftt-entity-index]
          +--rw ftt-entity-index             uint8
          +--rw ftt-entity-ds
          |  +--ro ftt-max-inputs?              uint8
          |  +--rw ftt-num-inputs?              uint8
          |  +--ro ftt-max-dtsfs?               uint8
          |  +--rw ftt-num-dtsfs?               uint8
          |  +--ro ftt-dtsf-max-inputs?         uint8
          |  +--rw ftt-inputs-to-tsf* [tsf-instance-number]
          |  |  +--rw tsf-instance-number    uint8
          |  |  +--rw assigned-inputs?       uint16
          |  +--rw ftt-ptp-instance-to-input* [ftt-input-index]
          |  |  +--rw ftt-input-index       uint8
          |  |  +--rw ptp-instance-index?   ptp-instance-ref
          |  +--rw ftt-hyst* [ftt-input-index]
          |  |  +--rw ftt-input-index    uint8
          |  |  +--rw hyst-list* [ftt-input-index]
          |  |     +--rw ftt-input-index    uint8
          |  |     +--rw hyst-value?        ptp-tt:time-interval
          |  +--rw ftt-max-as* [ftt-input-index]
```

```
|  |  +--rw ftt-input-index    uint8
|  |  +--rw max-as-list* [ftt-input-index]
|  |     +--rw ftt-input-index    uint8
|  |     +--rw max-as-value?      ptp-tt:time-interval
|  +--rw ftt-sel-change-thresh* [tsf-instance-number]
|  |  +--rw tsf-instance-number    uint8
|  |  +--rw change-threshold?      ptp-tt:time-interval
|  +--rw ftt-rr-thresh?               uint32
|  +--rw ftt-rr-thresh-std-dev?       uint32
|  +--rw ftt-use-nq* [tsf-instance-number]
|  |  +--rw tsf-instance-number    uint8
|  |  +--rw use-nq?                boolean
|  +--rw ftt-use-ftt-clk?             boolean
|  +--ro ftt-tsf-sel-input* [tsf-instance-number]
|  |  +--ro tsf-instance-number         uint8
|  |  +--ro selected-ftt-input-index?   uint8
|  +--ro ftt-sel-instance?            uint32
|  +--ro ftt-tsf-sel-change-cnt* [tsf-instance-number]
|  |  +--ro tsf-instance-number    uint8
|  |  +--ro sel-change-cnt?        uint16
|  +--ro ftt-input-trust-state* [ftt-input-index]
|  |  +--ro ftt-input-index          uint8
|  |  +--ro ftt-input-trust-state?   ftt-input-trust-state-type
|  +--ro ftt-trust-state?             ftt-output-trust-state-type
|  +--ro ftt-state-change-cnt?        uint16
+--rw ftt-entity-description-ds
   +--rw user-description?   string
```

17.6 YANG modules[7, 8]

Insert 17.6.3 at the end of 17.6 as follows:

17.6.3 Module ieee802-dot1as-ftt.yang

```
module ieee802-dot1as-ftt {
  yang-version 1.1;
  namespace "urn:ieee:std:802.1AS:yang:ieee802-dot1as-ftt";
  prefix dot1as-ftt;

  import ieee1588-ptp-tt {
    prefix ptp-tt;
  }

  organization
    "IEEE 802.1 Working Group";
  contact
    "WG-URL: http://www.ieee802.org/1/
     WG-EMail: stds-802-1-L@ieee.org
     Contact: IEEE 802.1 Working Group Chair
     Postal: C/O IEEE 802.1 Working Group
     IEEE Standards Association
```

[7] Copyright release for YANG modules: Users of this standard may freely reproduce the YANG modules contained in this subclause so that they can be used for their intended purpose.

[8] ASCII versions of the YANG modules are attached to the PDF version of this standard, and can be obtained by Web browser from the IEEE 802.1 Website at https://1.ieee802.org/yang-modules/.

```
    445 Hoes Lane
    Piscataway, NJ 08854
    USA
    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management objects that control fault-tolerant timing with
    time integrity (FTT) entities as specified in
    IEEE Std 802.1ASed-2026.

    References in this YANG module are to IEEE Std 802.1AS-2025
    as amended by IEEE Std 802.1ASed-2026.

    Copyright (C) IEEE (2026).
    This version of this YANG module is part of IEEE Std 802.1AS;
    see the standard itself for full legal notices.";

  revision 2026-02-11 {
    description
      "Published as part of IEEE Std 802.1ASed-2026.
       Initial version.";
    reference
      "IEEE Std 802.1AS - Timing and Synchronization for
       Time-Sensitive Applications: IEEE Std 802.1AS-2025,
       IEEE Std 802.1ASed-2026.";
  }

  feature ftt {
    description
      "This feature indicates that the device supports the FTT
       service.";
  }

  typedef ptp-instance-ref {
    type leafref {
      path "/ptp-tt:ptp/ptp-tt:instances/ptp-tt:instance"
         + "/ptp-tt:instance-index";
    }
    description
      "This type is used by data models that need to reference
       interfaces.";
  }

  typedef ftt-output-trust-state-type {
    type enumeration {
      enum NOT-TRUSTED {
        value 0;
        description
          "Integrity of time synchronization function is not
           achieved.";
      }
      enum TIME-TRUSTED {
        value 1;
        description
          "Integrity of time synchronization is achieved.";
      }
      enum FREQ-TRUSTED {
        value 2;
        description
          "Only the integrity of the rate-of-change of time is
```

```
          achieved.";
      }
      enum INIT {
        value 3;
        description
          "System is waiting to be invoked and has not determined a
           trust state.";
      }
    }
    description
      "The fttOutputTrustState type is an enumerated value that
       holds the trust state of the FTT entity.";
    reference
      "20.3.2.1 of IEEE Std 802.1AS.";
  }

  typedef ftt-input-trust-state-type {
    type enumeration {
      enum INACTIVE {
        value 0;
        description
          "The FTT input is not considered to be providing time
           values.";
      }
      enum ACTIVE {
        value 1;
        description
          "The FTT input is actively providing time values.";
      }
      enum TRUSTED {
        value 2;
        description
          "The FTT input is actively providing trusted time
           values.";
      }
      enum UNTRUSTED {
        value 3;
        description
          "The FTT input is actively providing untrusted time
           values.";
      }
    }
    description
      "The fttInputTrustState type is an enumerated value that
       holds the trust state of an FTT input.";
    reference
      "20.3.2.2 of IEEE Std 802.1AS.";
  }


  typedef uint48 {
    type uint64 {
      range "0..281474976710655";
    }
    description
      "Unsigned 48-bit integer.";
  }

  grouping fault-tolerant-timing-group {
```

```
    description
      "Management of a single FTT entity.";
    reference
      "14.23 of IEEE Std 802.1AS.";
    leaf ftt-max-inputs {
      type uint8 {
        range "0..16";
      }
      config false;
      description
        "Maximum number of input ClockTarget interfaces available
         on the FTT entity.";
      reference
        "14.23.2 of IEEE Std 802.1AS.";
    }
    leaf ftt-num-inputs {
      type uint8;
      description
        "The number of input ClockTarget interfaces currently
         enabled on the FTT entity.";
      reference
        "14.23.3 of IEEE Std 802.1AS.";
    }
    leaf ftt-max-dtsfs {
      type uint8 {
        range "0..126";
      }
      config false;
      description
        "Maximum number of DTSF instances available in the FTT
         entity.";
      reference
        "14.23.4 of IEEE Std 802.1AS.";
    }
    leaf ftt-num-dtsfs {
      type uint8;
      description
        "Number of DTSF instances enabled in the FTT entity.";
      reference
        "14.23.5 of IEEE Std 802.1AS.";
    }
    leaf ftt-dtsf-max-inputs {
      type uint8 {
        range "0..16";
      }
      config false;
      description
        "Maximum number of input ClockTarget interfaces available
         on each DTSF instance in the FTT entity.";
      reference
        "14.23.6 of IEEE Std 802.1AS.";
    }
    list ftt-inputs-to-tsf {
      key "tsf-instance-number";
      description
        "Mapping of FTT entity input ClockTarget interface index
         numbers to TSF instance numbers.";
      reference
        "14.23.7 of IEEE Std 802.1AS.";
```

```
      leaf tsf-instance-number {
        type uint8;
        description
          "The TSF instance number that the FTT entity's input
           index number is connected to.";
        reference
          "20.2.3 of IEEE Std 802.1AS.";
      }
      leaf assigned-inputs {
        type uint16;
        description
          "The bit map of FTT inputs that are assigned to the TSF
           instance. The most-significant bit corresponds to input
           index 16, the least-significant bit corresponds to input
           index 1. The number of inputs assigned to any DTSF
           instance (i.e., TSF instance with tsf-instance-number of
           1 or higher) cannot be greater than fttDtsfMaxInputs.";
        reference
          "14.23.7 of IEEE Std 802.1AS.";
      }
    }
    list ftt-ptp-instance-to-input {
      key "ftt-input-index";
      description
        "Mapping of PTP Instance index numbers to FTT entity input
         index numbers.";
      reference
        "14.23.8 of IEEE Std 802.1AS.";
      leaf ftt-input-index {
        type uint8;
        description
          "The FTT entity's input index number.";
        reference
          "20.2.1 of IEEE Std 802.1AS.";
      }
      leaf ptp-instance-index {
        type ptp-instance-ref;
        description
          "The instanceIndex number of the PTP Instance that is
           connected to the FTT input.";
        reference
          "14.1.1 of IEEE Std 802.1AS-2020.";
      }
    }
    list ftt-hyst {
      key "ftt-input-index";
      description
        "Hysteresis values to be added to fttMaxAs for pairs of
         input ClockTarget interfaces.";
      reference
        "14.23.9 of IEEE Std 802.1AS.";
      leaf ftt-input-index {
        type uint8;
        description
          "The first input index number.";
        reference
          "20.2.1 of IEEE Std 802.1AS.";
      }
      list hyst-list {
```

```
      key "ftt-input-index";
      description
        "Hysteresis values for pairs of input ClockTarget
         interfaces.";
      reference
        "14.23.9 of IEEE Std 802.1AS.";
      leaf ftt-input-index {
        type uint8;
        description
          "The second input index number.";
        reference
          "20.2.1 of IEEE Std 802.1AS.";
      }
      leaf hyst-value {
        type ptp-tt:time-interval;
        default "0";
        description
          "Hysteresis value to be added to fttMaxAs for the pair
           of input interfaces.";
        reference
          "14.23.9 of IEEE Std 802.1AS.";
      }
    }
  }
  list ftt-max-as {
    key "ftt-input-index";
    description
      "Maximum expected skew between pairs of input ClockTarget
       interfaces.";
    reference
      "14.23.10 of IEEE Std 802.1AS.";
    leaf ftt-input-index {
      type uint8;
      description
        "The first input index number.";
      reference
        "20.2.1 of IEEE Std 802.1AS.";
    }
    list max-as-list {
      key "ftt-input-index";
      description
        "The second input index number.";
      reference
        "14.23.10 of IEEE Std 802.1AS.";
      leaf ftt-input-index {
        type uint8;
        description
          "The second input index number.";
        reference
          "20.2.1 of IEEE Std 802.1AS.";
      }
      leaf max-as-value {
        type ptp-tt:time-interval;
        default "0";
        description
          "Maximum expected skew between the pair of input
           interfaces when those time values are not faulty.";
        reference
          "14.23.10 of IEEE Std 802.1AS.";
```

```
      }
    }
  }
  list ftt-sel-change-thresh {
    key "tsf-instance-number";
    description
      "Time difference change threshold used by TSF instances.";
    reference
      "14.23.11 of IEEE Std 802.1AS.";
    leaf tsf-instance-number {
      type uint8;
      description
        "The TSF instance number.";
      reference
        "20.2.3 of IEEE Std 802.1AS.";
    }
    leaf change-threshold {
      type ptp-tt:time-interval;
      description
        "The TimeInterval threshold used by the TSF instance.";
      reference
        "14.23.11 of IEEE Std 802.1AS.";
    }
  }
  leaf ftt-rr-thresh {
    type uint32;
    default "0";
    description
      "Maximum rate-ratio offset magnitude deemed acceptable for
       FREQ_TRUSTED state.";
    reference
      "14.23.12 of IEEE Std 802.1AS.";
  }
  leaf ftt-rr-thresh-std-dev {
    type uint32;
    default "0";
    description
      "Maximum standard deviation of rate-ratio offset deemed
       acceptable for FREQ_TRUSTED state.";
    reference
      "14.23.13 of IEEE Std 802.1AS.";
  }
  list ftt-use-nq {
    key "tsf-instance-number";
    description
      "Specifies whether a TSF instance uses the NQ input when no
       trusted time is available.
       If ftt-use-nq is TRUE, the use of the NQ input is enabled.
       If ftt-use-nq is FALSE, the use of the NQ input is not
       enabled. The default value is implementation-specific.";
    reference
      "14.23.14 and 20.2.4 of IEEE Std 802.1AS.";
    leaf tsf-instance-number {
      type uint8;
      description
        "The TSF instance number.";
      reference
        "20.2.3 of IEEE Std 802.1AS.";
    }
```

```
      leaf use-nq {
        type boolean;
        description
          "Whether the TSF instance uses the NQ input.";
        reference
          "14.23.14 of IEEE Std 802.1AS.";
      }
    }
    leaf ftt-use-ftt-clk {
      type boolean;
      description
        "Defines whether the FTT_CLK frequency is used as a
         reference for time integrity.
         If fttUseFTTClk is TRUE, the FTT_CLK frequency is used as
         a reference for checking time integrity.
         If fttUseFTTClk is FALSE, the FTT_CLK frequency is not
         used as a reference for checking time integrity.
         The default value is implementation-specific.";
      reference
        "14.23.15 of IEEE Std 802.1AS.";
    }
    list ftt-tsf-sel-input {
      key "tsf-instance-number";
      config false;
      description
        "The index number of the FTT input selected by each TSF
         instance.";
      reference
        "14.23.16 and 20.3.2.3 of IEEE Std 802.1AS.";
      leaf tsf-instance-number {
        type uint8;
        description
          "The TSF instance number.";
        reference
          "20.2.3 of IEEE Std 802.1AS.";
      }
      leaf selected-ftt-input-index {
        type uint8;
        description
          "FTT input index that is selected by the TSF instance.";
        reference
          "14.23.16 of IEEE Std 802.1AS.";
      }
    }
    leaf ftt-sel-instance {
      type uint32;
      config false;
      description
        "Instance index of the PTP Instance selected as the trusted
         time source.";
      reference
        "14.23.17 of IEEE Std 802.1AS.";
    }
    list ftt-tsf-sel-change-cnt {
      key "tsf-instance-number";
      config false;
      description
        "The number of times each TSF has changed its selection.";
      reference
```

```
        "14.23.18 of IEEE Std 802.1AS.";
      leaf tsf-instance-number {
        type uint8;
        description
          "The TSF instance number.";
        reference
          "20.2.3 of IEEE Std 802.1AS.";
      }
      leaf sel-change-cnt {
        type uint16;
        description
          "The number of times the TSF instance has changed its
           selection.";
        reference
          "14.23.18 of IEEE Std 802.1AS.";
      }
    }
    list ftt-input-trust-state {
      key "ftt-input-index";
      config false;
      description
        "Trust state of all FTT inputs.";
      reference
        "14.23.19 of IEEE Std 802.1AS.";
      leaf ftt-input-index {
        type uint8;
        description
         "The FTT input index.";
        reference
          "14.23.16 of IEEE Std 802.1AS.";
      }
      leaf ftt-input-trust-state {
        type ftt-input-trust-state-type;
        description
          "Trust state of the FTT input.";
        reference
          "14.23.19 of IEEE Std 802.1AS.";
      }
    }
    leaf ftt-trust-state {
      type ftt-output-trust-state-type;
      default "NOT-TRUSTED";
      config false;
      description
        "The trust state of the FTT entity.";
      reference
        "14.23.20 and 20.3.2.1 of IEEE Std 802.1AS.";
    }
    leaf ftt-state-change-cnt {
      type uint16;
      default "0";
      config false;
      description
        "Number of times the FTT entity has changed its trust
         state.";
      reference
        "14.23.21 of IEEE Std 802.1AS.";
    }
  }
```

```
  augment "/ptp-tt:ptp/ptp-tt:common-services" {
    description
      "Augment IEEE Std 1588 commonServices with ftt-service.";
    container ftt-service {
      if-feature "ftt";
      description
        "The FTT service structure contains the fttEntityList,
         which is a list of instances of the FTT service.";
      reference
        "14.23 of IEEE Std 802.1AS.";
      list ftt-entity {
        key "ftt-entity-index";
        description
          "Indexed list of FTT entities in the FTT service.";
        leaf ftt-entity-index {
          type uint8;
          description
            "Index of the FTT entity.";
        }
        container ftt-entity-ds {
          description
            "The fttEntityDS describes the attributes of the
             respective instance of the FTT service.";
          reference
            "14.23 of IEEE Std 802.1AS.";
          uses fault-tolerant-timing-group;
        }
        container ftt-entity-description-ds {
          description
            "The fttEntityDescriptionDS contains descriptive
             information for the respective instance of the FTT
             service.";
          reference
            "14.24 of IEEE Std 802.1AS.";
          leaf user-description {
            type string {
              length "0..128";
            }
            description
              "Configuration description of the Fault-Tolerant
               Timing entity.";
            reference
              "14.24.2 of IEEE Std 802.1AS.";
          }
        }
      }
    }
  }
}
```

Insert Clause 20 in numeric order as follows:

20. Fault-tolerant timing with time integrity

20.1 Overview

20.1.1 General

To achieve fault-tolerant timing with time integrity, this standard defines an FTT entity. The concepts used by the FTT entity are described in 20.1.2. An overview of the FTT entity is given in 20.1.3 and details on the FTT entity are given in 20.2.

20.1.2 Fault-tolerant time synchronization concepts

20.1.2.1 Availability of time

The continuous availability of time is enhanced by redundancy. For gPTP, this redundancy can be implemented by using multiple time domains, multiple time distribution paths, and multiple PTP Instances in Bridges and end stations.

20.1.2.2 Trust and integrity of time

For this standard, a trusted time is one that passes a specified criterion that identifies it as being within acceptable bounds. This establishment of trust gives integrity to the time.

For gPTP, trust, and hence integrity, is established through the comparison of the times coming from independent time sources and the observation that they match within the specified criterion. See H.4 for an example of such a criterion.

20.1.2.3 Time distribution

Time distribution, per this standard, is the process of distributing the time from time source nodes (GMs) to timeReceivers (PTP Instances) in Bridges and end stations over gPTP communication paths.

20.1.2.4 Dependent times

Dependent times share one or more common time-influencing components, which includes a common GM, continuously synchronized GMs, GMs that share a common (continuously connected) ClockSource, or a common PTP Relay Instance.

Because dependent times share one or more common influencers, they do not, on their own, enable end-to-end integrity checking of the time synchronization function. However, they can be used to improve the availability of a given time source and can provide partial integrity checks. For example, an application that receives timing from a single GM through more than one redundant synchronization tree (and PTP Instances) has increased availability of that GM’s time and can check the integrity of the synchronization trees by comparing the time received from the respective PTP Instances. However, because the time originates from a single GM, the integrity of that GM’s time cannot be confirmed and, thus, end-to-end integrity of the time synchronization function is not achieved.

Dependent times can be identified by checking if they meet any the following criteria:

— They have the same gPTP domainNumber. This indicates that gPTP messages from the same PTP GM are received by two PTP End Instances, on two distinct physical ports, that are serviced by the FTT entity.
— They have different gPTP domainNumbers but the same gmTimeBaseIndicator. This indicates that the gPTP messages come from different PTP GMs that share the same ClockSource.
— They have gPTP domainNumbers that are defined by a management entity to be dependent. For example, the management entity could define gPTP time inputs at an FTT entity as dependent if their respective gPTP communication paths share a common PTP Relay Instance or a common Bridge.

20.1.2.5 Independent times

Independent times do not share any common time-influencing components with each other and, therefore, deliver independent time values. Independent gPTP times are, by definition, from different domains.

Because independent times do not share any common influencers, they can enable end-to-end integrity checking of the time synchronization function, provided their times track sufficiently closely. The clock sources of GMs providing independent times need to be aligned to each other in a manner that is resilient to faults (see J.1). Because the independent times provide common time, the inherent redundancy can also be used to improve the availability of the time synchronization function.

When a set of dependent times are used in combination with a set of independent times, the dependent times can be reduced to a single independent time and used to achieve end-to-end integrity of the time synchronization function. This operation is performed by the FTT entity. See H.3 for a discussion on balancing availability and integrity with FTT entities.

20.1.3 Model of operation for fault-tolerant timing with time integrity

An FTT entity enables fault-tolerant timing with time integrity. The FTT entity is located between a ClockTarget entity (Clause 9) and one or more PTP Instances of a time-aware system. The FTT entity manages the selection of a time source from amongst two or more PTP Instances to support increased availability and integrity. The input to an FTT entity, hereafter called simply FTT input is the ClockTargetEventCapture.result (see 9.3.3) from the PTP Instances connected to the FTT entity. The output of an FTT entity, hereafter called simply FTT output, is the ClockTargetEventCapture.result of the PTP Instance selected by the FTT entity. The FTT entity also supports the use of a single PTP Instance (single input) but, with a single input, it does not provide any enhancements for increased availability or for integrity. The effect of the FTT entity's selection is limited to the time-aware system in which it operates. Figure 20-1 illustrates the FTT entity operating with three PTP Instances.

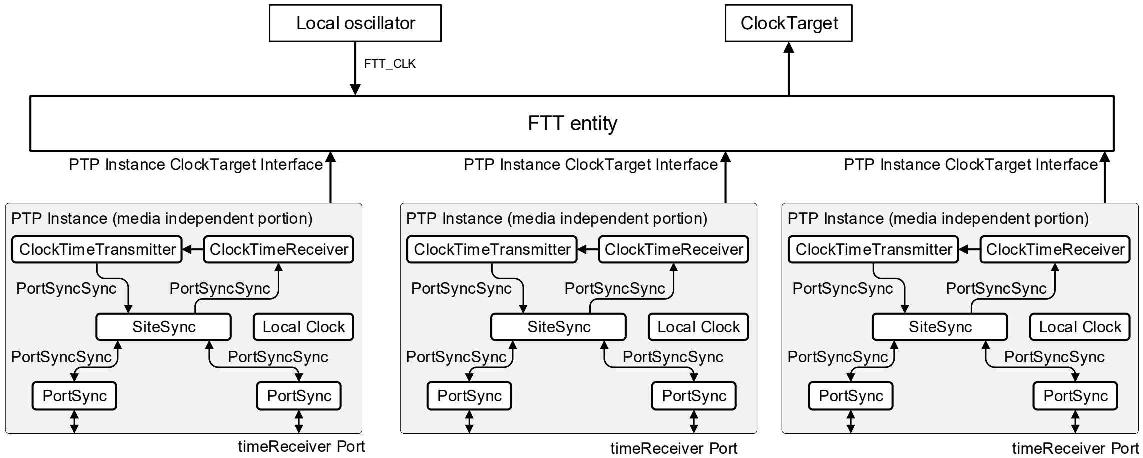


Figure 20-1 — FTT entity in operation

20.1.3.1 Assumptions

The following list provides the detailed assumptions and goals for the FTT entity:

— A fault-tolerant network (with time integrity) and its configuration are static during normal operation.
— All gPTP ports are configured using the external port configuration provision (i.e., the BTCA is not used).
— There is no administrative reconfiguration during run-time in the event of faults.
— While operation with one PTP Instance (single input) is supported by the FTT entity, two or more PTP Instances (multiple inputs) are required for fault-tolerant timing with time integrity.
— gPTP times are recognized as being dependent or independent as defined in 20.1.2.4 and 20.1.2.5, respectively.

20.2 FTT entity

A functional block diagram of the FTT entity is shown in Figure 20-2.

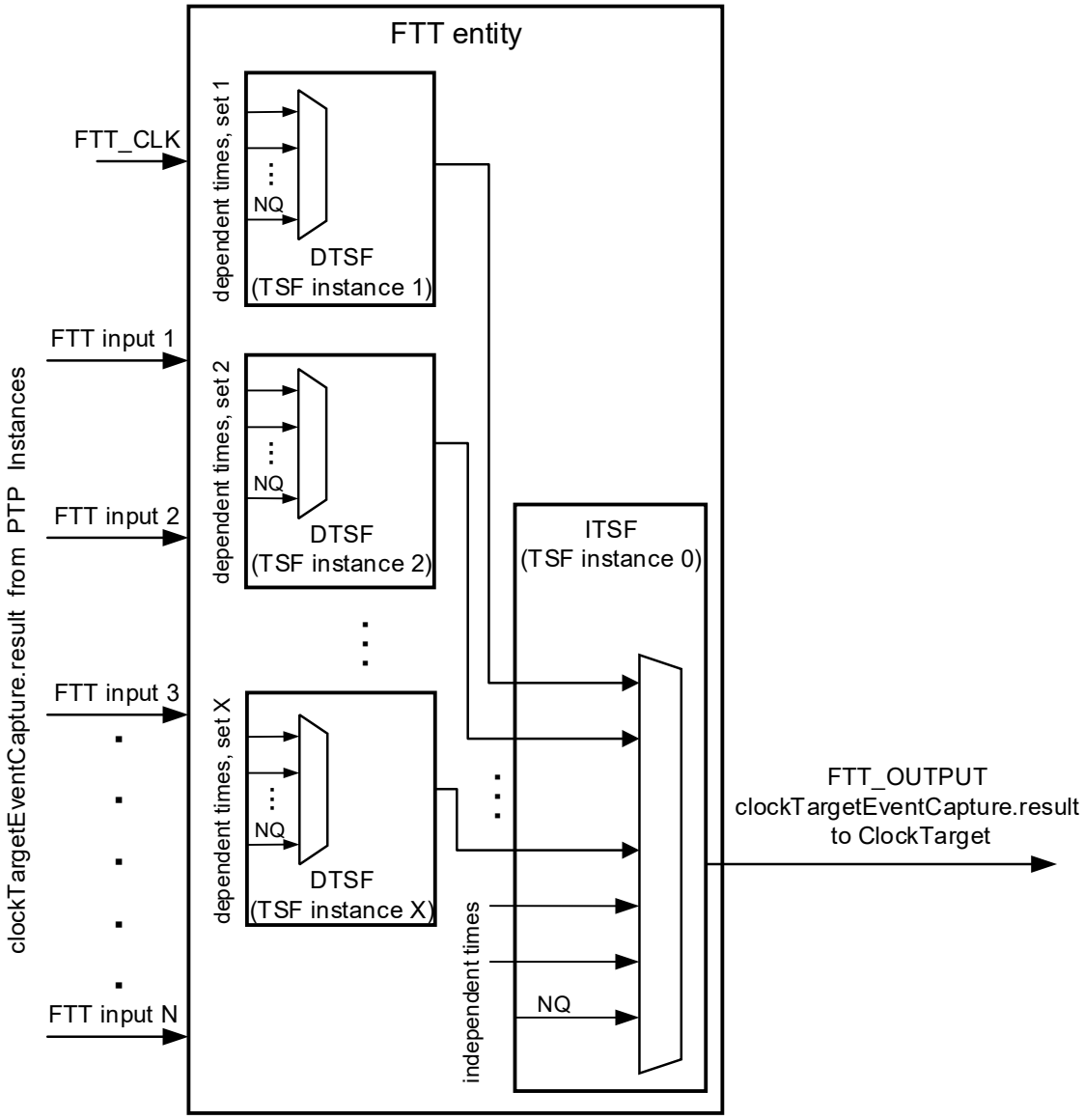


Figure 20-2 — FTT entity functional block diagram

20.2.1 FTT input interfaces

The input interfaces to the FTT entity, hereafter referred to as FTT inputs, are the ClockTargetEventCapture interface of the PTP Instances connected to the FTT entity. They provide time values from the PTP Instances to the time selection function (TSF, see 20.2.3) instances of the FTT entity. The FTT inputs are indexed (FTT input index) for ease of reference. Each input to the FTT entity is also accompanied by the instanceIndex number of the corresponding PTP Instance. The managed object fttPtpInstanceToInput provides the mapping between the PTP Instance index and the FTT input index.

The FTT inputs provide either dependent times or independent times to the FTT entity. FTT inputs that provide dependent times are referred to as dependent inputs and FTT inputs that provide independent times are referred to as independent inputs.

The FTT entity uses the ClockTargetEventCapture interface type (see 9.3). The use of other ClockTarget interface types with the FTT entity is discussed in H.5.

20.2.2 FTT clock

An FTT entity can use a free running clock (FTT_CLK) as an additional source of timing. If fttUseFTTClk is TRUE, the FTT entity uses a free-running clock (FTT_CLK) as a potential source of timing information for establishing trust (20.3). FTT_CLK is independent of the time values being received by the PTP Instances that are connected to the FTT entity. The health and trust of FTT_CLK is outside the scope of this standard.

20.2.3 Time selection function

An FTT entity includes one or more TSF instances, each referenced by a TSF instance number. Each TSF instance analyzes its set of inputs (ClockTarget interfaces), determines which corresponding time values can be trusted and selects one of the trusted time values using a time selection algorithm. The output of the TSF instance is set to the selected input and the corresponding PTP Instance.

The inputs to a TSF may provide either dependent times or independent times.

One TSF instance in the FTT entity is used for selecting a time from a set of independent times (see 20.1.2.5). This TSF instance is called the independent time selection function (ITSF) and is assigned the TSF instance number of 0.

If any of the inputs provide dependent times to the FTT entity (see 20.1.2.4), then one TSF instance is used to select a time for each set of dependent times. These TSF instances are called dependent time selection functions (DTSFs) and are assigned TSF instance numbers from 1 to fttNumDtsfs (see 14.23.5).

Therefore, an FTT entity includes zero or more DTSF instances and exactly one ITSF instance. The TSF instance number is used to uniquely reference a particular TSF instance.

The time-index selection algorithm for the TSF is the mid-value time selection algorithm (MVTSA, see 20.4). The MVTSA shall be supported if the FTT entity is supported.

The TSF processes are described in 20.3.3.4.

20.2.4 Not qualified (NQ) input

When none of the inputs to a TSF instance are determined to be trusted and the TSF instance's fttUseNQ parameter is TRUE, the TSF instance selects its NQ input as the TSF output. The NQ input has an input index of 255. The NQ input contains the ClockTargetEventCapture interface parameters from any one (selection is implementation specific) of the inputs being processed, but with the isSynced status and the gmPresent status set to FALSE, to indicate the untrusted condition.

NOTE—Because the time provided by the NQ input is, by definition, not trusted, the values of its other parameters (aside from isSynced and gmPresent) have no functional impact.

All parameters in the NQ time are listed below:

— domainNumber = the domainNumber from the selected input
— timeReceiverTimeCallback = the timeReceiverTimeCallback value from the selected input
— isSynced = FALSE
— gmPresent = FALSE
— errorCondition = the errorCondition value from the selected input

— clockPeriod = the clockPeriod value from the selected input
— timeReceiverCallbackPhase = the timeReceiverCallbackPhase value invoked by the ClockTarget entity
— grandmasterIdentity = the grandmasterIdentity value from the selected input
— gmTimeBaseIndicator = the gmTimeBaseIndicator value from the selected input
— lastGmPhaseChange = the lastGmPhaseChange value from the selected input
— lastGmFreqChange = the lastGmFreqChange value from the selected input

20.2.5 FTT output interface

The FTT entity's output is the ClockTargetEventCatpure.result from the PTP Instance selected by the ITSF instance in the FTT entity. The FTT entity accompanies the output ClockTargetEventCatpure.result with the instanceIndex number of the selected PTP Instance.

The FTT entity provides a fault-tolerant time to a ClockTarget entity via an output ClockTargetEventCapture application interface (FTT_OUTPUT). The instanceIndex number of the PTP Instance associated with the output ClockTargetEventCapture application interface is available via the fttSelInstance management object (see 14.23.17).

20.3 FTT state machine

20.3.1 General

The FTT entity state machine specified in 20.3 interacts with all the TSFs (instantiated in the FTT entity's DTSF instances and in the FTT entity's ITSF instance) to find and select a trusted input, if one exists, to present as the FTT entity's output.

The TSF process is described in 20.3.3.4. The mid-value time selection algorithm (MVTSA) used by the TSF instances is described in 20.4.

20.3.2 FTT state machine global variables

20.3.2.1 fttTrustState:

The trust state of the FTT entity. The value is taken from the enumeration in Table 20-1.

Table 20-1—fttTrustState enumeration

Value	State	Specification
0	NOT-TRUSTED	The input selected by the FTT entity does not meet the specified criteria to establish trust and the conditions to use the FTT_CLK are not satisfied. In this state, integrity of time synchronization function is not achieved.
1	TIME-TRUSTED	The FTT entity has selected an input whose time value meets the specified criterion to establish trust. In this state, integrity of time synchronization is achieved.
2	FREQ-TRUSTED	The FTT entity has selected an input whose time value does not meet the specified criteria to establish trust, but the conditions for using the FTT_CLK to maintain a trusted status are satisfied. In this state, only the integrity of the rate-of-change of time is achieved.
3	INIT	Initialization after the FTT entity powers on and is enabled. In this state, the system is waiting to be invoked and has not determined a trust state.

20.3.2.2 fttInputTrustState:

An Enumeration2 array of length fttNumInputs. fttInputTrustState[*j*], where *j* is a value from 1 to fttNumInputs, is the trust state of the FTT input (see 20.2.1) with index *j*. The trust state of the FTT input takes a value from the enumeration in Table 20-2.

Table 20-2—fttInputTrustState enumeration

Value	State	Specification
0	INACTIVE	Either the isSynced or the gmPresent value of the FTT input is FALSE. In this state, the FTT input is not considered to be providing time values.
1	ACTIVE	Both the isSynced and gmPresent values of the FTT input are TRUE. In this state, the FTT input is actively providing time values but has not yet determined to be TRUSTED or UNTRUSTED.
2	TRUSTED	The FTT input is ACTIVE and the provided time values meet the criteria to establish trust (see 20.4.1). In this state, the FTT input is providing trusted time values.
3	UNTRUSTED	The FTT input is ACTIVE and the provided time values do not meet the criteria to establish trust (see 20.4.1). In this state, the FTT input is providing untrusted time values.

20.3.2.3 fttTsfSelInput:

An UInteger8 array of length fttNumDtsfs + 1 (see 14.23.5). fftTsfSelInputs[*j*] is the index number of the FTT input selected by TSF instance with instance number *j*. The values range from 1 to fttNumInputs and the value of 255, which represents the NQ input (see 20.2.4).

20.3.2.4 itsfSelInput:

The index number of the FTT input selected by the ITSF instance. The values range from 1 to fttNumInputs and the value of 255, which represents the NQ input (see 20.2.4). The data type for itsfSelInput is UInteger8.

20.3.2.5 itsfTrustState:

Enumerated trust state of the ITSF instance. It is the value of fttInputTrustState[itsfSelInput] (see 20.3.2.2).

20.3.2.6 rRatioOff:

Offset of the ratio between the rate of the time provided on the output of the ITSF instance to the rate of the FTT entity's local clock, FTT_CLK. The value is expressed as a fractional frequency offset multiplied by 2^{41}, that is:

$$rRatioOff = (|\text{rate of time at the output of ITSF} / \text{rate of FTT\_CLK} - 1.0|) \times 2^{41}$$

The magnitude of this offset is one of the considerations used by the FTT entity's state machine to determine whether it enters the FREQ_TRUSTED state or the NOT_TRUSTED state. See 20.3.4. The data type for rRatioOff is unsigned 32-bit integer.

NOTE—The rate measurement period is application dependent.

20.3.2.7 rRatioSdOff:

The standard deviation of the offset of the ratio between the rate of the time provided on the output of the ITSF instance to the rate of the FTT entity's local clock, FTT_CLK. It is one of the considerations used by the FTT entity's state machine to determine whether it enters the FREQ_TRUSTED state or the NOT_TRUSTED state. See 20.3.4. The data type for rRatioSdOff is unsigned 32-bit integer.

NOTE—The rate measurement period is application dependent.

20.3.2.8 fttCtecPtr:

An array of pointers of length fttNumInputs. fttCtecPtr[*j*] is the pointer to a structure whose members contain the values of the parameters of a ClockTargetEventCapture.result primitive from the FTT input with index *j*.

20.3.3 FTT state machine functions

20.3.3.1 setCtec(N):

Creates a structure whose parameters contain the fields of ClockTargetEventCapture.result (see 9.3.3), populates the fields with values from the FTT input (of index N), and returns a pointer to this structure.

20.3.3.2 updateItsfInputs:

Appends the inputs selected by all enabled DTSFs, fttTsfSelInput[1:fttNumDtsfs] to the ITSF input set, fttInputsToTsf[0].

20.3.3.3 updateStateChangeCnt(newState):

The updateStateChangeCnt procedure updates the fttStateChangeCnt (see 14.23.21) based on the input argument (newState) and the current state of the fttTrustState object as follows:

If the newState is different from the fttTrustState, then increment fttStateChangeCnt by 1. The fttStateChangeCnt rolls over to 0 if the count is incremented when its current value is 65 535. The default value is 0.

20.3.3.4 tsf(tsfNum):

Selects one of the inputs from the set of FTT inputs mapped to the TSF with instance number tsfNum via the global variable fttInputsToTsf and sets the output of the TSF to the global variable fttTsfSelInput (14.23.16), as follows:

a) If the number of inputs to the TSF is 1, then the only available input, fttInputsToTsf[tsfNum], is set as the selected input in the fttTsfSelInput[tsfNum] variable.
b) If the number of inputs to the TSF is more than 1, call the MVTSA with the TSF instance number (tsfNum) and set the selected input, fttTsfSelInput[tsfNum], to the returned value (input index) from MVTSA if all of the following conditions are met:
   — MVTSA returns a different input than the previously selected input to the TSF, fttTsfSelInput[tsfNum].
   — The difference in time value of the newly selected input and the previously selected input is greater than the fttSelChangeThresh defined for this instance of TSF or the trust state of the newly selected and the previously selected inputs differ from each other.

If a new input is selected by the TSF as per item b), increment fttTsfSelChangeCnt[tsfNum] by 1.

NOTE—If an iteration of the TSF function does not result in the selection of a (new) input as per items a) or b), the selected input of the corresponding DTSF or ITSF does not change.

The time selection function is invoked by each DTSF instance and by the ITSF instance to select an input from the set of available inputs. The TSF function utilizes a time selection algorithm to select one of the inputs and sets the fttTsfSelInput object for the corresponding TSF instance. The time selection algorithm, MVTSA, is described in 20.4.

20.3.3.5 resetTsfStatistics(tsfNum)

The resetTsfStatistics procedure sets the fttTsfSelChangeCnt value to zero for all TSF instances.

20.3.4 FTT state diagram

The FTT entity state machine shall implement the function specified by the state diagram in Figure 20-3, the variables specified in 20.3.2, and the functions specified in 20.3.3.

The FTT_INITIALIZE state of the FTT entity state machine resets the fttTsfSelChangeCnt and fttStatusChangeCnt management objects to zero and the fttTrustState global variable to INIT. This prepares these objects for further updates by this state machine. The state machine transitions unconditionally to the WAIT_INVOKE state.

The WAIT_INVOKE state of the FTT entity state machine waits for a ClockTargetEventCapture.invoke event from the ClockTarget. When the event is received, the state machine transitions to the GET_INPUTS state.

The GET_INPUTS state of the FTT state machine simultaneously sends a ClockTargetEventCapture.invoke event to all the PTP Instances connected to the FTT entity. Once all the PTP Instances respond by providing their ClockTargetEventCapture.result, this state transitions to the SAVE_CLKTARGET_RSLT state.

The SAVE_CLKTARGET_RSLT state calls the setCtec() procedure for each of the FTT entity’s input ClockTargetEventCapture interfaces. For each FTT input, the setCtec() procedure returns a pointer to the structure containing the fields of the ClockTargetEventCapture.result from the corresponding PTP Instance. These pointers are saved in the fttCtecPtr global variable, indexed by the FTT input index.

If any DTSF instances are enabled in the FTT entity, the state machine transitions to the DTSF_SELCT state. If no DTSF instances are enabled in the FTT entity, the state machine transitions to the ITSF_SELECT state.

The DTSF_SELECT state calls the TSF function for every enabled DTSF instance in the FTT entity. This causes every DTSF instance to run its selection procedure and produce an updated output that is saved in the fttTsfSelInput object. After all DTSFs complete their selection process, the updateItsfInput function is called to add the inputs selected by each DTSF to the ITSF input set. The state machine transitions unconditionally to the ITSF_SELECT state.

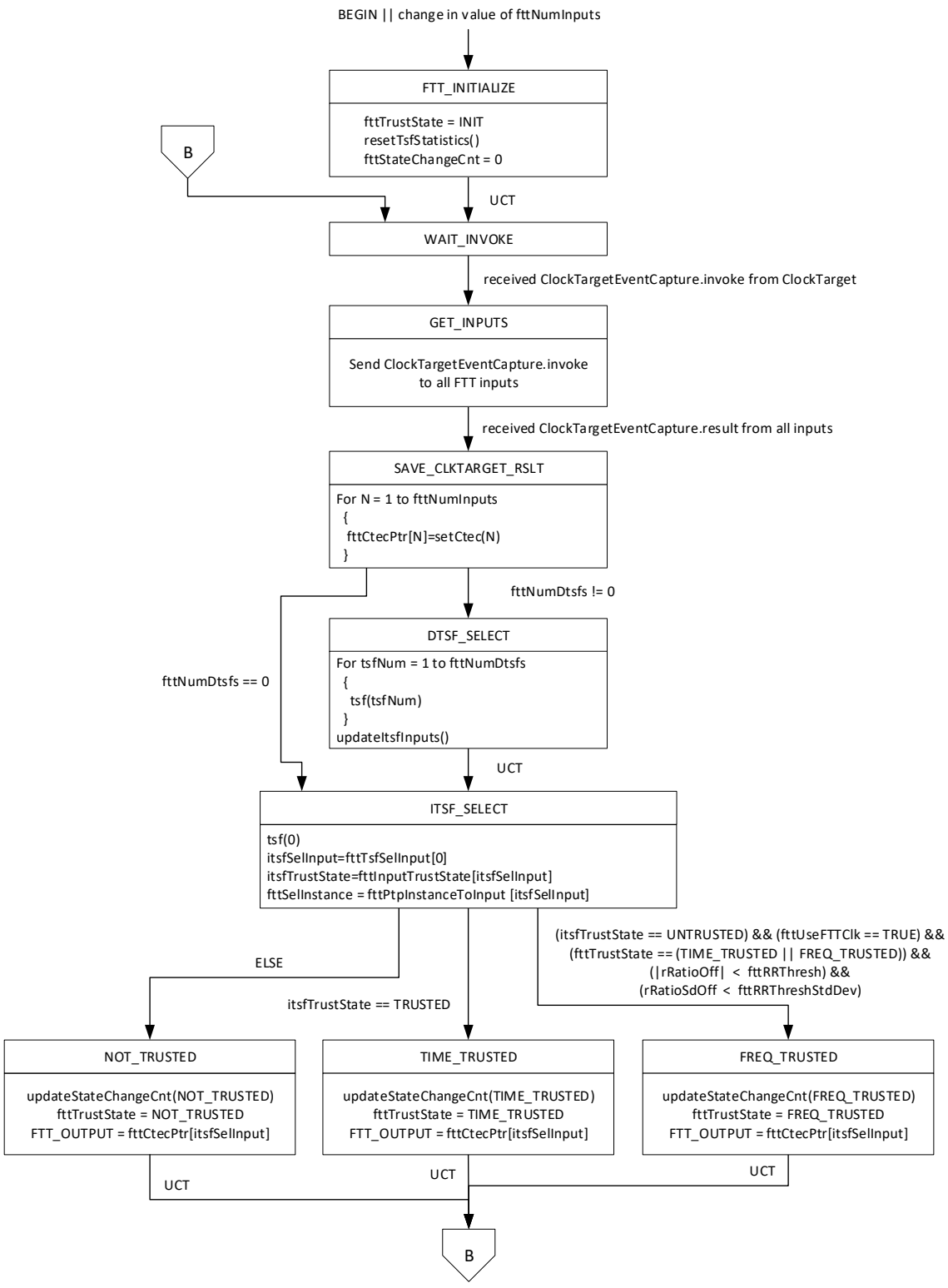


Figure 20-3 — FTT entity state machine

The ITSF_SELECT state calls the TSF function for the ITSF instance, which causes the ITSF instance to run its selection procedure and produce an updated output, and then performs corresponding updates to the itsfSelInput variable and the itsfTrustState and fttSelInstance management objects. If the trust state of the ITSF instance's selected input is TRUSTED, the state machine transitions to the TIME_TRUSTED state. If the trust state of the ITSF instance's selected input is NOT_TRUSTED but the conditions for using the FTT_CLK to maintain a trusted status are satisfied (i.e., the FTT entity was previously in the TIME_TRUSTED state or the FREQ_TRUSTED state and the time from its previously selected input is progressing at a rate and with a rate variation that are within configured thresholds), the state machine transitions to the FREQ_TRUSTED state. Otherwise, the state machine transitions to the NOT_TRUSTED state.

The NOT_TRUSTED, TIME_TRUSTED, and FREQ_TRUSTED states all call the updateStateChangeCnt procedure to update the ftttStateChangeCnt object, set the fttTrustState object to the corresponding state, and update the ClockTargetEventCapture.result output of the FTT entity, on FTT_OUTPUT, with the contents of the selected input ClockTargetEventCapture interface. The state machine transitions unconditionally to the WAIT_INVOKE state.

The FREQ_TRUSTED state calls the updateStateChangeCnt procedure to update the ftttStateChangeCnt object, sets the fttTrustState object to FREQ_TRUSTED, and updates the ClockTargetEventCapture.result output of the FTT entity, on FTT_OUTPUT, with the contents of the selected input ClockTargetEventCapture interface. The state machine transitions unconditionally to the WAIT_INVOKE state.

The NOT_TRUSTED state calls the updateStateChangeCnt procedure to update the ftttStateChangeCnt object, sets the fttTrustState object to NOT_TRUSTED, and updates the ClockTargetEventCapture.result output of the FTT entity, on FTT_OUTPUT, with the contents of either the NQ time or the selected input ClockTargetEventCapture interface. The state machine transitions unconditionally to the WAIT_INVOKE state.

20.4 Mid-value time selection algorithm

The mid-value time selection algorithm is used to determine which inputs have trusted time values and then find, amongst those trusted inputs, the input with the mid-value time. The MVTSA is the default algorithm called by the TSF Function (20.3.3.4).

The MVTSA looks at all possible combinations of input pairs to determine which pairs have time values that satisfy their corresponding specified maximum accepted skew magnitude threshold, fttMaxAs (see 14.23.10). An input with a time value that satisfies the corresponding fttMaxAs threshold with one or more other inputs is considered to be trusted. The input that has the mid-value time amongst all the trusted inputs is selected as the output of the MVTSA. If the number of trusted time indexes is even, the selected time index is the one with the smaller index value.

MVTSA(tsfNum): Selects the mid-value time from trusted inputs of a TSF and returns the index of the selected input or the index of NQ input by executing the following three procedures (20.4.1, 20.4.2, and 20.4.3) steps in order.

20.4.1 setInputTrustState(tsfNum):

Determines the trust state of each input assigned to the TSF with index tsfNum via the global variable fttMapIndextoTsf and sets the state value for the corresponding input in the global variable fttInputTrustState as follows:

a) If both the isSynced and gmPresent value of an input are TRUE, the trust state for the corresponding input is set to ACTIVE in global variable fttInputTrustState. Otherwise, the trust state of the input is set to INACTIVE.

b) A pair-wise comparison of time values from all ACTIVE inputs is conducted to determine if the inputs are TRUSTED or UNTRUSTED. The trust state of an ACTIVE input, with fttInputIndex of *x*, is set to TRUSTED in fttInputTrustState if one of the following two conditions is TRUE and is set to UNTRUSTED if both of the following two conditions are FALSE.

   1) The fttInputTrustState[*x*] was previously UNTRUSTED and its current time value is within the maximum expected skew, fttMaxAS[*x*][*y*], of at least one other input's (index *y*) time value.

   2) The fttInputTrustState[*x*] was previously TRUSTED and its current time value is within the maximum expected skew plus hysteresis (fttMaxAs[*x*][*y*] + fttHyst[*x*][*y*]) of at least one other input's (index *y*) time value.

NOTE 1—The hysteresis in item b) enables the use of one skew level to assert the trust status and another skew level to de-assert the trust status. This can prevent superfluous changes in the trust state of two inputs (with input index *x* and *y*) when the time values of those inputs are close to the fttMaxAs[*x*][*y*] threshold. The value of this hysteresis is to be determined by the user of the FTT entity.

NOTE 2—The method for storing the previous trust state of FTT inputs is implementation dependent.

20.4.2 setSelectionPool():

Creates a candidate pool of time values to be considered for selection by the selectionFunction procedure based on the fttInputTrustState established by the setInputTrustState procedure, as follows:

a) If any of the inputs were determined to be TRUSTED, then the selection pool consists of the time values of all the inputs that were determined to be TRUSTED.

b) If none of the inputs were determined to be TRUSTED and fttUseNQ[tsfNum] is TRUE, then the selection pool consists of only the time value from "NQ" (see 20.3.2.3).

c) If none of the inputs were determined to be TRUSTED and fttUseNQ[tsfNum] is FALSE and at least one input was determined to be ACTIVE, the selection pool consists of the time values from all the inputs that were determined to be ACTIVE.

d) If none of the inputs were determined to be TRUSTED and fttUseNQ[tsfNum] is FALSE and none of the inputs were determined to be ACTIVE, the selection pool consists of the time values from all the inputs.

NOTE—When fttUseNQ[tsfNum] is FALSE [items c) and d)], the corresponding TSF instance does not select the NQ input even in the absence of a trusted or active input. This allows a TSF instance to select either an untrusted active or inactive input as its output when no trusted or active inputs are found.

20.4.3 selectionFunction():

Selects one of the time values from the selection pool to be returned by the MVTSA function as follows:

- All the time values in the selection pool are ordered from smallest (i.e., earliest) to largest (i.e., latest).
- If the ordered list of time values contains an odd number of candidates, the middle value is selected.
- If the ordered list of time values contains an even number of candidates, one of the two middle values is selected based on the index number (fttInputIndex) associated with the input providing the corresponding time value. The candidate with the lower input index number is selected.
- If the selection pool has a single entry, the lone value is selected.

The MVTSA returns the index number of the input whose time value is selected by the selectionFunction procedure.

Annex A

(normative)

Protocol Implementation Conformance Statement (PICS) proforma[9]

A.5 Major capabilities

Insert a new row at the end of the table in A.5 as follows (unchanged rows not shown):

Item	Feature	Status	References	Support
…				
FTT Entity	Does the time-aware system implement the FTT entity as specified in 20.2 through 20.4?	O	5.3, 9.3, 14.23, 17.6.3, 20.1.3	Yes [ ] No [ ]

A.19 Remote management

Insert a new row at the end of the table in A.19 as follows (unchanged rows not shown):

Item	Feature	Status	References	Support
…				
RMGT-6	If a remote management protocol that supports YANG is listed in RMGT-2, and if the FTT entity is supported, is the YANG data model ieee802-dot1as-ftt of Clause 17 supported?	RMGT:O	5.3, Clause 17, 20.1.3	Yes [ ] No [ ] N/A[ ]

[9] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

Insert new Annex H, Annex I, and Annex J after Annex G as follows and re-number the existing Annex H as Annex K:

Annex H

(informative)

Fault-tolerant time synchronization

H.1 Introduction

It is important for some time-sensitive applications (e.g., aerospace, automotive, and industrial automation) to consider fault tolerance and integrity for the time synchronization function, to enable reliable and trustworthy system behavior. For example, an aerospace network is typically expected to tolerate one or more arbitrary faults in Bridges, end stations, links, and GMs while maintaining continuous availability and integrity for the time synchronization function.

The typical timing requirement scenarios are summarized below:

a) **High availability timing:** This scenario requires availability of the time for a high percentage of a service period. A fault can introduce an interruption to this availability, but this interruption is short in duration relative to the corresponding service period.
b) **Fault-tolerant timing:** This scenario requires continuous availability of the time, even in the presence of a fault or, in some scenarios, multiple faults.
c) **Fault-tolerant timing with time integrity:** This scenario requires continuous availability of a trusted time, even in the presence of a fault or, in some scenarios, multiple faults.

Features of gPTP, as defined in this standard, can be used to support all three of the above timing requirement scenarios. The use of these features can enable system designers to meet their applications' timing requirements for assured systems.

For the high availability timing scenario, this standard supports multiple time distribution paths and potential reorganization of the synchronization trees, via the BTCA, to overcome faults. The BTCA is, however, not compatible with some applications that have stringent fault-tolerance requirements. First, a reorganization of the synchronization tree to overcome a fault takes time to complete. Depending on the size of the network and where the fault occurred, there might be a high degree of variability in the fault recovery time of the BTCA. Second, many fault-sensitive applications use engineered networks with static configuration and deterministic behavior and, thus, cannot use the BTCA.

For the fault-tolerant timing scenario, this standard supports multiple time domains, multiple GMs, multiple time distribution paths, multiple PTP Instances per port in Bridges and end stations, and the external port configuration mode (10.3.1.3), which allows static time distribution paths to be established. These features provide active redundancy, enabling the redundant function to take over when the original function fails.

For the fault-tolerant timing with time integrity scenario, this standard supports the features listed under the fault-tolerant timing scenario and adds support for the FTT entity functionality. The FTT entity uses multiple domains with multiple (potentially redundant) synchronization trees to provide configurable and deterministic fault recovery behavior, to determine the trustworthiness of the times that it is receiving, and to select a trusted time for use by its clock target.

H.2 Fault-tolerance claims for the FTT entity

The fault-tolerance and integrity establishment claims that can be made for the FTT entity are given in this annex. The concepts upon which these claims are based are described in 20.1.2.2.

Table H-1—Fault-tolerance claims of the FTT entity

Fault condition in FTT inputs	Fault detection	Fault tolerance
Independent times with uncorrelated faults	Yes, for all cases	Yes, if there is at least one pair of non-faulty independent FTT inputs
Independent times with correlated[a] faults	Yes, if there are $(2n + 1)$ independent FTT inputs for n faults	Yes, if there are $(2n + 1)$ independent FTT inputs for n faults
Dependent times with correlated[b] faults	Yes, detected at ITSF	Yes, if there are $(2n + 1)$ independent inputs to the ITSF for n independent faults

[a] Independent times with correlated faults is a special case used for illustration purposes only. In practice, this condition would only happen by chance and would not be maintained long-term.
[b] This scenario assumes the failure occurs on the time-influencing component that is common to the FTT inputs that carry the dependent times.

For the first fault condition (i.e., independent times with uncorrelated faults) in Table H-1, any faulty FTT input, x, would not match within the fttMaxAS[x][y] skew limit between it and any other FTT input, y, regardless of whether FTT input y is faulty or non-faulty. Thus, any faulty FTT input can be identified. Even if there is just one pair of non-faulty FTT inputs amongst many faulty FTT inputs, this pair alone would be deemed to be trustworthy and one FTT input from the pair would be selected by the FTT entity for its output.

For the second fault condition (i.e., independent times with correlated faults), the exceptional circumstance of a faulty FTT input having an error that matches the error of another faulty FTT input is considered. While this circumstance can happen by chance, this correlated failure would not be expected to be maintained given the FTT inputs do not share any common time influencing components. Under this circumstance, more non-faulty FTT inputs are needed to override the faulty FTT inputs that have a correlated error.

The third fault condition (i.e., dependent times with correlated faults) in Table H-1 is a typical failure scenario amongst dependent times because they share a common time-influencing component. These dependent times are processed by a DTSF. The DTSF produces an output with the correlated fault that is used by the ITSF. At the ITSF level, this correlated fault is detected when the time value at the DTSF output is compared to other time values at the ITSF inputs.

H.3 Balancing availability and integrity

The following compromises to fault-tolerant timing and time integrity are common in many implementations:

— Use of a common clock source for all GMs.
— Use of only two domains instead of three or more.
— An insufficient number of independent paths from the GMs to the FTT entities.

For the first listed compromise, some applications only need the time-aware components of the local system to be synchronized to the arbitrary timescale of the local system and the integrity of this timescale does not need to be protected. In this scenario, a single clock source running off a local oscillator might be sufficient to satisfy the time agreement and preservation requirements of the system and be used as the source for all the GMs in the system. An example network topology with this compromise is discussed in J.2.2.

For the second listed compromise, some applications have a fail-stop requirement instead of a fail-operational requirement. A fail-operational requirement means the application has to continue operating correctly even if a fault (or up to *N* faults) has been detected. A fail-stop requirement means the application stops operation once a fault is detected. Only two independent times are needed to satisfy a fail-stop requirement, and this can be achieved using two domains. Example network topologies with this type of compromise are discussed in J.2.1, J.2.2, and J.2.3.

For the third listed compromise, it can be difficult and perhaps even impossible to create independent paths from all GMs to every FTT entity in a network. For these networks, the FTT entity can still support increased availability and integrity scenario for fail-stop applications if there are at least two independent paths from two GMs to the FTT entity (see the discussion about Figure J-5 in J.2.3). Otherwise, if there are no independent paths from any of the GMs, then the FTT entity is only able to support increased availability but not integrity (see 20.3). Example network topologies with this compromise are discussed in J.2.3 and J.2.4.

The availability and integrity scenarios supported by the FTT entity are listed below:

- Multiple time inputs, all of which are independent:
  - This scenario operates with multiple domains all processed by the ITSF.
  - In this mode, the FTT entity supports increased availability and integrity.
- Multiple time inputs, some of which are dependent with each other and some of which are independent with each other:
  - This scenario operates with multiple domains.
  - The dependent time inputs are processed together in a DTSF.
  - The independent time inputs and the outputs of every DTSF are processed by the ITSF.
  - In this mode, the FTT entity supports increased availability and integrity.
- Multiple time inputs, all of which are dependent with each other:
  - This scenario operates with a single domain or with multiple domains.
  - All time inputs share a common time-influencing component and, thus, can suffer from a common-mode failure.
  - In this mode, the FTT entity supports increased availability and potential time distribution integrity, but not GM integrity.
- Single time input:
  - This scenario operates with a single time input and a single domain.
  - In this mode, the FTT entity does not support increased availability or integrity.

The application is expected to be aware of the availability and integrity limitations based on the scenario in which the FTT entity is operating.

H.4 Time error accumulation example

H.4.1 General

As gPTP time is distributed through a network from a GM to PTP Relay Instances and/or PTP Instances, time error accumulates. Some of the reasons for this time error accumulation are listed below:

— Timing errors at the GM (TEgm)
— Timing errors at intermediate PTP Relay Instances (TErly)
— Timing errors at the PTP End Instance (TEend)
— Link asymmetry between PTP Instances (TElnk)

The above time errors are illustrated in Figure H-1.

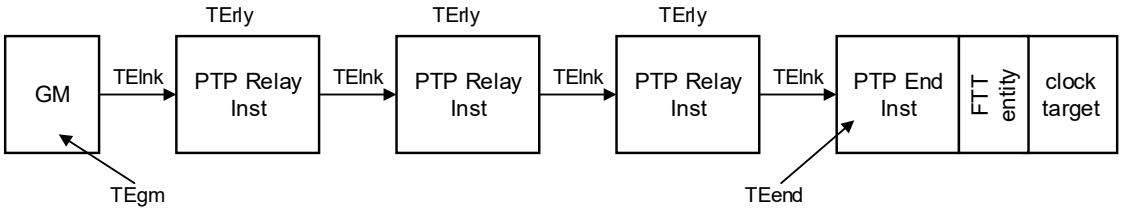


Figure H-1—Time error accumulation across a network

It is possible to determine the potential maximum absolute value of each of the above time errors and, thus, the maximum potential time error at the PTP End Instance. This result, maxAccumTE (see H.4.2), if available, can be used by the FTT entity in its selection process for a trusted gPTP time to present to the ClockTarget.

NOTE—The potential maximum absolute values of TEgm, TErly, TEend, and TElnk are expected to be calculated or measured prior to operational usage. This could be done at the design, characterization, or certification phase. The specific methods and procedures to do this are not defined in this standard.

The ENHANCED_ACCURACY_METRICS TLV from IEEE Std 1588a-2023 [B10a] can be used to accumulate the maximum constant and dynamic time errors of each PTP Instance and the connecting links, on a hop-by-hop basis, in the path from the GM to, but not including, the final PTP End Instance. This TLV is carried in Announce messages.

Time skew between two time distribution paths is shown in Figure H-2. The time error contributors across each path are the same as shown in Figure H-1, but the contributors on each path are marked with the path's identifying suffix, x or y.

The various time skew results are described in H.4.2, H.4.3, H.4.4, and H.4.5.

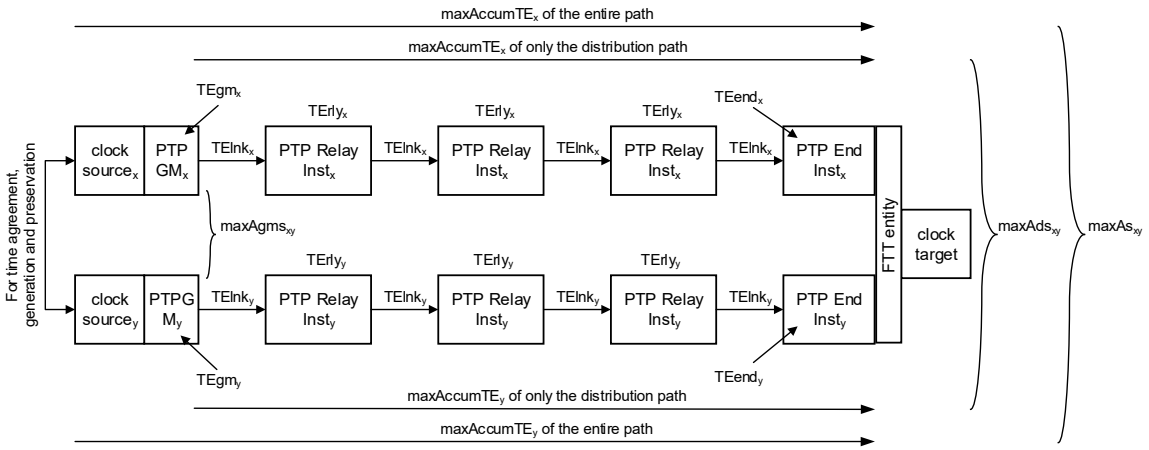


Figure H-2—Time skew across two time distribution paths

H.4.2 maxAccumTE$_x$

For the list of reasons for time error accumulation given in H.4.1, the parameter maxAccumTEx is the maximum non-faulty accumulated time error magnitude for the time distributed on path x, from its GM (TEgm), through all intermediate PTP Relay Instances (TErly) and the corresponding links (TElnk), to the PTP End Instance (TEend) that is connected to the FTT entity.

$$maxAccumTE_x = max(|TEgm_x|) + \sum max(|TErly_x|) + \sum max(|TE\ln k_x|) + max(|TEend_x|)$$

Where Σ is the summation symbol and represents a summation of all instances of the term adjacent to it.

H.4.3 maxAgms$_{xy}$

The parameter maxAgms$_{xy}$ is the maximum accepted time skew magnitude between two non-faulty PTP GMs, GM$_x$ and GM$_y$. This value is equal to the worst-case time error magnitude between the two GMs when they are not faulty.

$$maxAgms_{xy} = max(|TEgm_x|) + max(|TEgm_y|)$$

H.4.4 maxAds$_{xy}$

The parameter maxAds$_{xy}$ is the maximum accepted distribution skew magnitude between the time of two non-faulty times, distributed on path x and path y. This value is equal to the worst-case time error magnitude between the two times, from the perspective of the FTT entity, resulting from their distribution paths when they are not faulty.

$$maxAds_{xy} = \sum max(|TErly_x|) + \sum max(|TE\ln k_x|) + max(|TEend_x|) + \sum max(|TErly_y|) + \sum max(|TE\ln k_y|) + max(|TEend_y|)$$

H.4.5 maxAs$_{xy}$

The parameter maxAs$_{xy}$ is the maximum accepted skew magnitude between two non-faulty times, distributed on path x and path y. This value is equal to the worst-case time error magnitude between two

synchronized times, from the perspective of the FTT entity, when they are not faulty. This value can be used as a criterion to determine the trustworthiness of the times being compared.

$$maxAs_{xy} = maxAgms_{xy} + maxAds_{xy}$$
$$= maxAccumTE_x + maxAccumTE_y$$

System integrators have to design their TSN network, which supports fault-tolerant timing and time integrity, such that synchronization-dependent nodes can withstand a drift equal to the magnitude of this maximum accepted skew.

H.5 Alternate ClockTarget interfaces

This annex discusses concepts for interworking alternate ClockTarget interfaces (e.g., ClockTargetTriggerGenerate and ClockTargetClockGenerator interfaces from 9.4 and 9.5, respectively) from a PTP Instance to a ClockTargetEventCapture interface (see 9.3) of the FTT entity and from the ClockTargetEventCapture interface of the FTT entity to a ClockTarget entity.

Figure H-3 shows an interworking function between PTP Instances, the FTT entity, and the ClockTarget entity, where the PTP Instances and the ClockTarget entity do not use the ClockTargetEventCapture interface. The interworking function has multiple proxy timers, each of which holds the PTP time that is recovered from a corresponding PTP Instance using the information provided by its alternate ClockTarget interface. These proxy timers are used as sources of time for the ClockTargetEventCapture interfaces to the FTT entity. The FTT entity's fttTrustState (see 14.23.20) and fttSelInstance (see 14.23.17) management objects are then used to select which proxy timer's time, or the NQ time (see 20.2.4) if trust is not found, is sent to a functional block that generates the alternate ClockTarget interface to the ClockTarget entity.

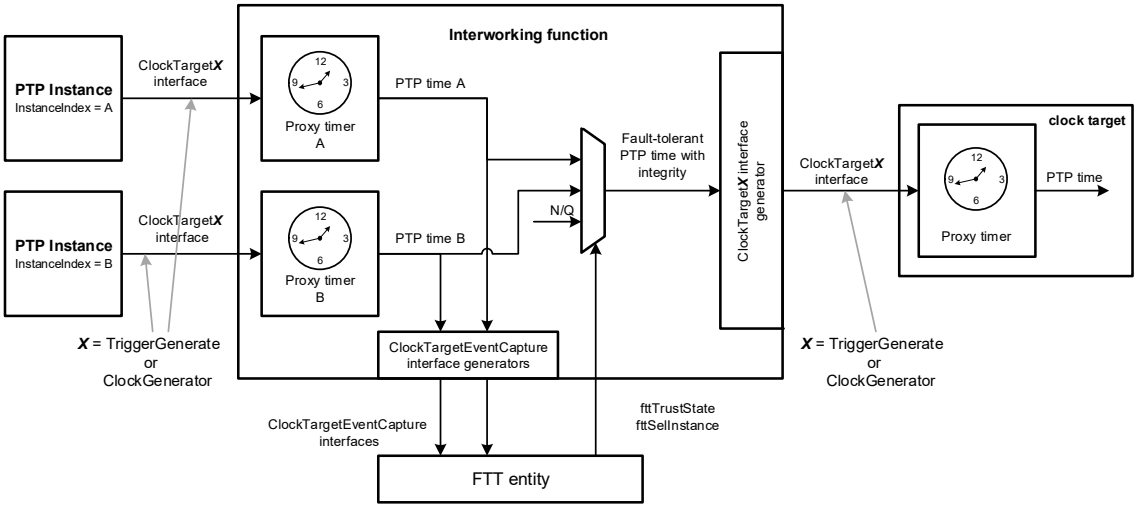


Figure H-3— Conceptual interworking function to alternate ClockTarget interfaces for the FTT entity

Annex I

(informative)

Time agreement generation and preservation

I.1 Overview

Time agreement generation and preservation is the process by which multiple time source nodes (e.g., clock sources for GMs) come to an agreement on the time and maintain that agreement in the presence of both faults and oscillator drift. This process preserves both the collective accuracy and relative precision of the resulting set of GMs.Time agreement generation and preservation is done in a manner that is resilient to faults as described in I.2. This annex provides time agreement generation and preservation examples.

I.2 Fault models

These implementations need to be tolerant to an occurrence or some occurrences of the following fault types, which include Byzantine faults. All implementations have to reduce the possibility and/or probability of occurrence for all of these fault types.

a) Value domain faults
   - Manifest: symmetric and asymmetric
   - Omissive: symmetric and asymmetric
   - Transmissive: symmetric and asymmetric

b) Time domain faults

Value domain faults are faults where the value of the information received by the intended user is incorrect.

Time domain faults are faults where the information is received by the intended user at an incorrect/unexpected time (e.g., inadvertent activations, out-of-order messages). These faults can sometimes be transformed into value-domain faults. For example, the reception of a correct value at the wrong time can be treated as an incorrect value at the right time.

Manifest faults are faults where the intended user of the information can discern that the information is incorrect (e.g., checksum error). These faults can be transformed into omissive faults.

Omissive faults are faults where the information is not received by the intended user, but the intended user might not be aware of this omission.

Transmissive faults are faults where incorrect information is received by the intended user, but the intended user can only discern that this information is incorrect by comparison with redundant sources.

Symmetric faults are faults that are the same for all the intended users of the information.

Asymmetric faults are faults that are different for some of or all the intended users of the information.

Byzantine faults are asymmetric. It is commonly accepted (see Doley et al. [B3a]) that at least $2N + 1$ clock sources and $3N + 1$ components (technically, fault-containment regions, see Hodson et al. [B6a]) are required to achieve tolerance for N Byzantine faults. This would be sufficient to achieve tolerance for N

faults of any of the types listed above. The examples shown in this annex have four components, each a clock source, and, thus, can tolerate one Byzantine fault.

It is not necessary for an FTT entity to have four input times to achieve time integrity for a Clock Target because, once time agreement is established at the GMs, only $2N + 1 = 3$ components are required to tolerate $N = 1$ Byzantine faults during time distribution. This is because no convergence operations are performed (i.e., no feedback) for time distribution and, thus, it does not have Byzantine manifestations.

I.3 Assumptions

Assumptions related to implementations of a time agreement generation and preservation function.

a) Fault-free oscillators have a specified bounded rate of drift with respect to time.
b) For initial synchronization, the cluster of clocks being subject to agreement generation and preservation satisfy the initial precision (i.e., the time source nodes all have the same unit of time, within a defined relative difference).
c) For preservation of synchronization (in the presence of faults):

— Each synchronization period begins with the time source nodes satisfying initial precision
— During the synchronization period, the time source nodes can drift within the bounds of the known oscillator drift rate
— During the synchronization period, the time source nodes exchange their time with other time source nodes (a small error can be introduced with this communication)
— Convergence function: The time source nodes compute a local adjustment using some fault-tolerant average of the readings from the other time source nodes
— This convergence function needs to satisfy the following, within some specified number of faults:
    — Precision enhancement: If the time source nodes satisfy initial precision at the beginning of the period, the adjusted values after computing and adjusting using the fault-tolerant average will again satisfy initial precision at the beginning of the next period (this implies some convergence property due to the computation of the fault-tolerant average).
    — Accuracy preservation: The computed averages, when applied, constrain the cluster of synchronized clocks to a bounded rate of drift with respect to an ideal time reference.

I.4 PTP-based time agreement generation and preservation

A PTP-based time agreement generation and preservation example is shown in Figure I-1.

In this example, PTP sync messages are broadcast from a PTP port, which is configured to be in the PASSIVE state (see Lamport and Melliar-Smith [B28a]), from each time agreement generation and preservation (TAGP) entity through a high integrity PTP message distributor with PTP Transparent Clock (see Lamport and Melliar-Smith [B28a]) capabilities to a PTP Port, which is also configured to be in the PASSIVE state, on all the other TAGP entities. These PTP sync messages and their corresponding departure times and arrival times are used by each TAGP entity to compare the frequency and time of its clock source against those of the other clock sources and to syntonize the frequency and synchronize the time of its clock source.

The integrity of the PTP message distributor is critical in this example because it is a common potential source of failure in the conveyance of the PTP information between the TAGP entities. This PTP message distributor needs to reduce its probability of creating false information while distributing the PTP sync messages. This could be done by adding redundancy and error checking to the PTP message distributor's timestamping, correctionField updating, and broadcasting functions.

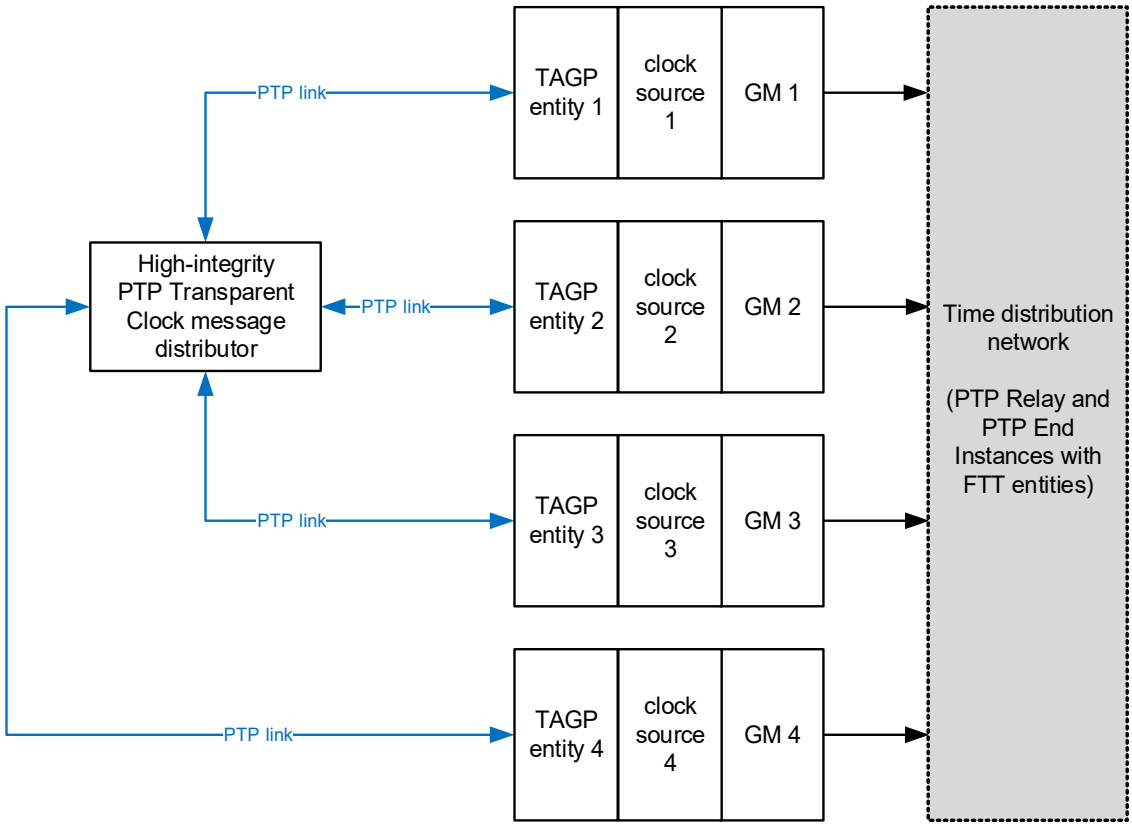


Figure I-1— PTP-based time agreement generation and preservation example with high-integrity message distributor

Another PTP-based time agreement generation and preservation example is shown in Figure I-2.

In the example of Figure I-2, point-to-point links are used to convey PTP sync messages from one TAGP entity to another TAGP entity. In comparison with the example of Figure I-1, there is no high-integrity message distributor but more PTP ports (also in PASSIVE state) are needed at each TAGP entity. Like in the example of Figure I-1, the PTP sync messages and their corresponding departure times and arrival times are used by each TAGP entity to compare the frequency and time of its clock source against those of the other clock sources and to syntonize the frequency and synchronize the time of its clock source.

For initial time agreement generation, it might be acceptable to assume that no fault exists in the time agreement functionality. Thus, each TAGP entity could use the PTP sync message timestamps to compare the frequency offsets of all the clock sources (including its own), and then tune its clock source's frequency to match the average of the frequencies of all clock sources. After syntonization of all clock sources is achieved, each TAGP entity can perform a similar compare, average, and alignment process to synchronize their times.

Once initial time agreement generation is achieved, the TAGP entities switch to a time agreement preservation mode. In this mode, each TAGP entity could continuously use the PTP sync message timestamps to compare the frequency and time offsets of all the clock sources (including its own), determine which match within an expected range, exclude those (including its own clock source) that do not meet this expectation, and then tune its clock source's frequency and/or time to match the average of the of all the non-excluded clock sources.

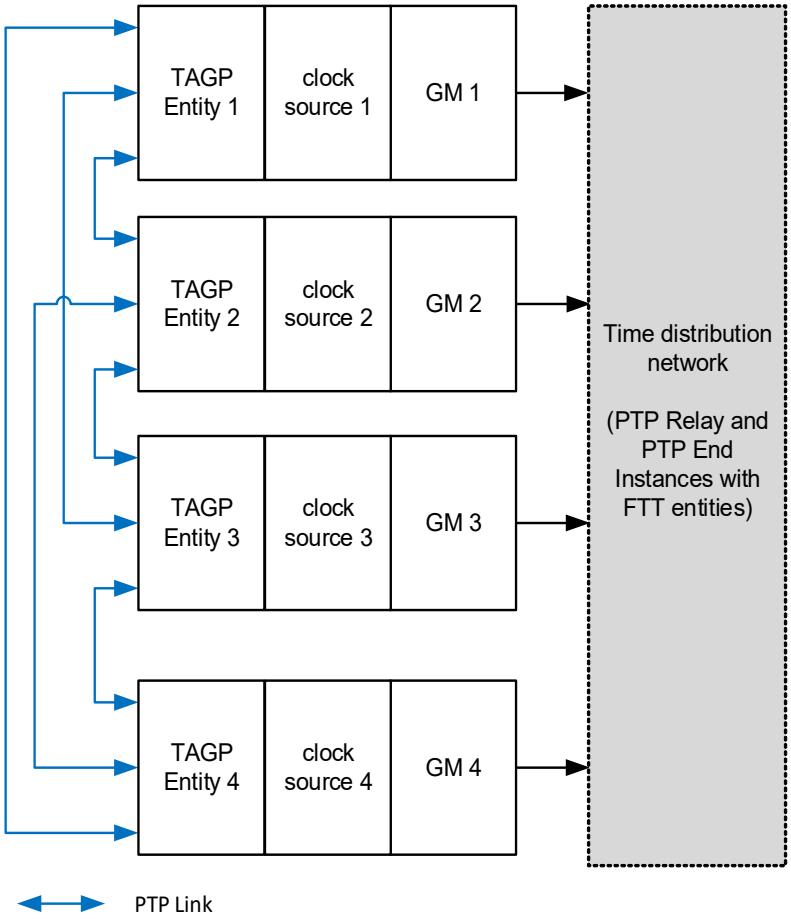


Figure I-2— PTP-based time agreement generation and preservation example without high-integrity message distributor

I.5 Other time agreement generation and preservation methods

Time agreement generation and preservation could be implemented without use of PTP.

Pulse per second (PPS) interfaces could be used instead of PTP interfaces to convey clock source frequency and time between the TAGP entities. A typical PPS interface includes the PPS event signal, which conveys the occurrence of a PPS event, and a UART interface, which conveys the time that corresponds to the PPS event.

Another method is to use GNSS as clock sources at each of the redundant GMs since the time provided by a GNSS has already achieved time agreement and preservation. Thus, its use as the clock source for a GM comes with this established benefit. However, the potential loss of GNSS signaling due to environmental conditions or jamming has to be considered for this use case.

Annex J

(informative)

FTT in systems (examples)

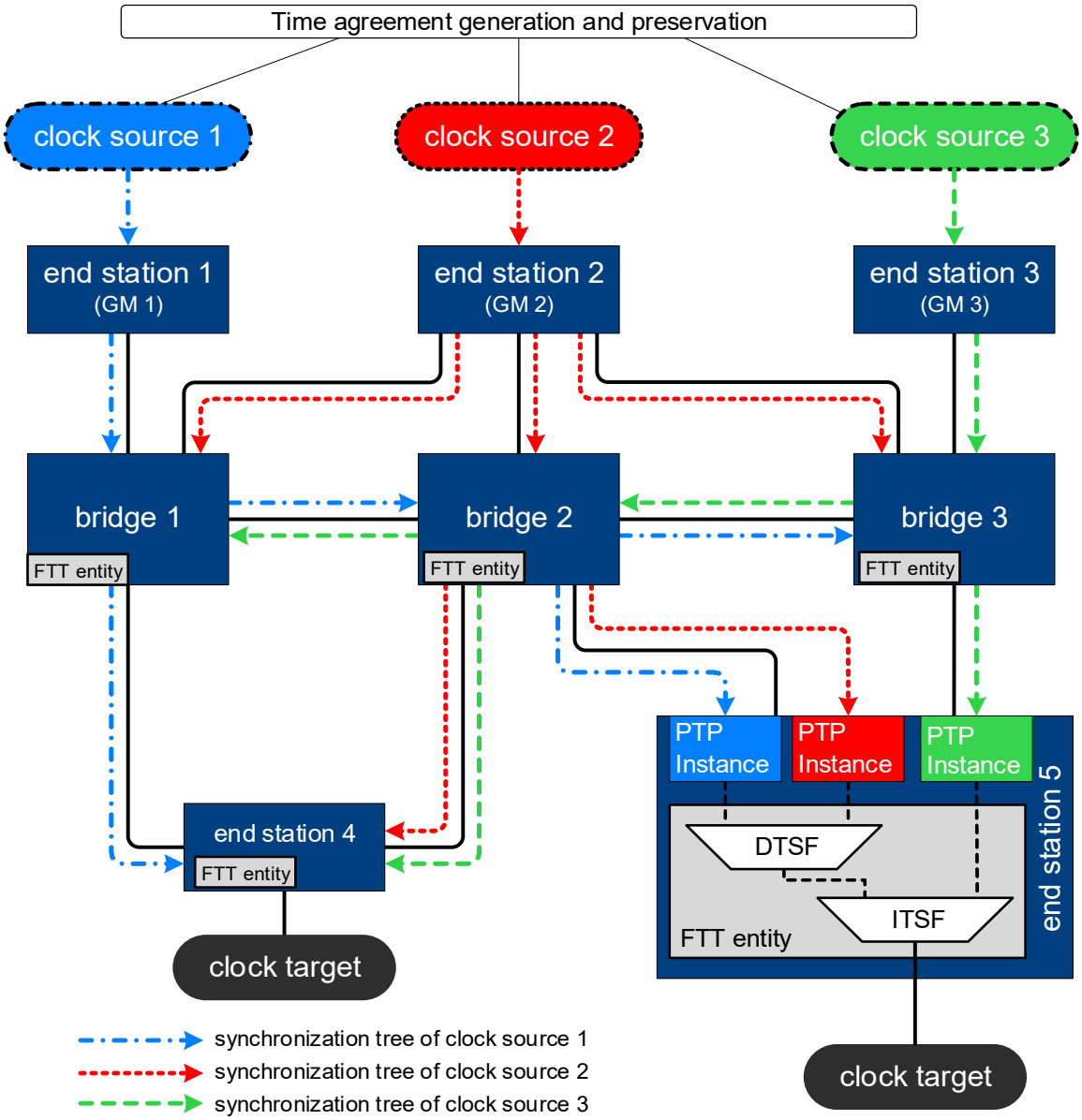
J.1 Clock domain management

The FTT entity defined in this standard uses multiple domains with multiple (potentially redundant) synchronization trees to provide configurable and deterministic fault recovery behavior and to support time integrity.

To illustrate how the FTT entity works in an example application, Figure J-1 shows a simple network using three clock sources distributing time through multiple time synchronization trees to Bridges and end stations that implement FTT entities to select a time source from the multiple domains. The existence of multiple overlapping timing domains presents a problem for the forwarding of scheduled traffic because each port can only forward traffic using one clock reference. The three clock sources in the example are therefore assumed to share a common time through implementation of a time agreement and preservation function, I.1, to allow the forwarding of time-aware traffic across domains. The time agreement and preservation function is implementation-specific and is not specified in this standard. The clock sources in the example figure synchronize three GMs in end stations 1, 2, and 3, that distribute time through the network on separate, but overlapping, synchronization trees. Bridges 1, 2, and 3 distribute time from the three GMs through the network to all Bridges and end stations where FTT entities select the best available clock to generate a local clock target that is used to support the enhancements for scheduled traffic (8.6.8.4 of IEEE Std 802.1Q-2022).

End station 5 shows three PTP Instances receiving time through the network and providing clock target interfaces to an FTT entity consisting of one DTSF instance and one ITSF instance. Clock sources 1 and 2 arrive at end station 5 through a common Bridge device, bridge 2, and are therefore considered to be dependent and require a DTSF instance to select the best clock source to provide this as an independent input to the ITSF instance along with the output of the PTP Instance that uses clock source 3. The time from clock source 3 arrives at end station 5 through a path that is independent from the path taken by clock sources 1 and 2, and can therefore be considered to be independent from these. The output of the ITSF instance is then used to generate the local clock target for end station 5.

As shown in the figure, each of the Bridges in the network and end station 4 also include FTT entities to generate local clock targets. FTT entities can be configured differently in each device in the network depending upon the relationships of the available clock sources to one another, and select a clock source to generate the local clock target that is used to support generation and forwarding of time-aware traffic. To simplify the figure, end stations 1, 2, and 3 are not shown as recipients of the synchronization trees from the other two clock sources and do not show FTT entities; however, in a real network application they might be expected to receive the other time signals and to contain FTT entities to support fault-tolerant traffic generation. Once the three clock sources have established time agreement and each of the FTT entities have selected the best clock at each of network nodes, then all of the clock targets will be synchronized and time-aware traffic from any end point in the network can be forwarded synchronously to any other end point through the available Bridges.



NOTE 1 — The methods to synchronize the ClockSources of all GMs are not specified in this standard.
NOTE 2 — All the “bridges” in this figure are examples of time-aware systems that contain Relay Instances and an FTT entity. The methods used to terminate time from PTP Relay Instances is not specified in this standard.
NOTE 3 — All the end stations in this figure are examples of time-aware systems that contain PTP End Instances and an FTT entity.

Figure J-1— Multiple domains with FTT entities

Under normal operating conditions, each of the nodes in the example network will receive time from each of the three domains and generate a local clock target by selecting the best clock source according to the configured FTT entity attributes. In the case of a fault in the network, an input becomes UNTRUSTED at a Bridge or at an end station and the corresponding FTT entity selects the best available time from its remaining inputs. For example, if the link between bridges 2 and 3 were to fail, the synchronization tree from clock source 3 would not propagate to bridges 1 and 2 or to end station 4, and the synchronization tree from clock source 1 would not propagate to bridge 3. In this case, bridge 3 would still receive times from clock sources 2 and 3, and end station 4 would still receive times from clock sources 1 and 2. The FTT entities at bridge 3 and end station 4 could still select between these inputs.

This simple example does not address time integrity. See H.2 for a discussion of integrity and availability.

J.2 FTT entity operation in example network topologies

The use of FTT entities in examples of commonly used network topologies is shown in this annex, with details on the potential enhancements to time availability and time integrity.

J.2.1 *N* x point-to-point networks

An example $N \times$ point-to-point network topology is shown in Figure J-2. This network topology provides N independent inputs to the FTT entity via N PTP instances.

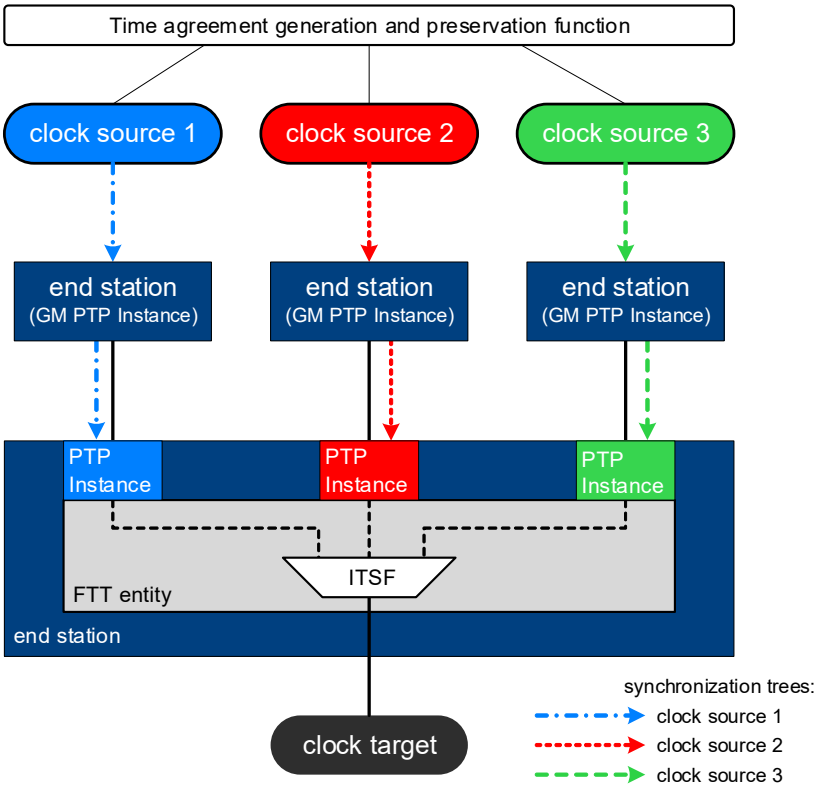


Figure J-2— FTT entity in example 3 × point-to-point network

In a scenario with $N = 2$, time availability with time integrity is achieved when there are no faults. Thus, a $2 \times$ point-to-point network topology would be suitable for fail-stop applications, which terminate operation when any failure is detected in the time integrity.

In the scenario of Figure J-2, with $N = 3$, time availability with time integrity is achieved when there is up to one fault. Thus, a 3 × point-to-point network topology would be suitable for fail-operational applications that can continue running when there is up to one fault.

J.2.2 Dual-homed network

An example dual-homed network is shown in Figure J-3. It provides two dependent inputs from one GM, through two PTP Instances, to the FTT entity. Because the two time inputs are dependent, they pass through a DTSF instance in the FTT entity. Because there are no independent time inputs to the FTT entity, the DTSF instance's output is the only connection to the ITSF instance.

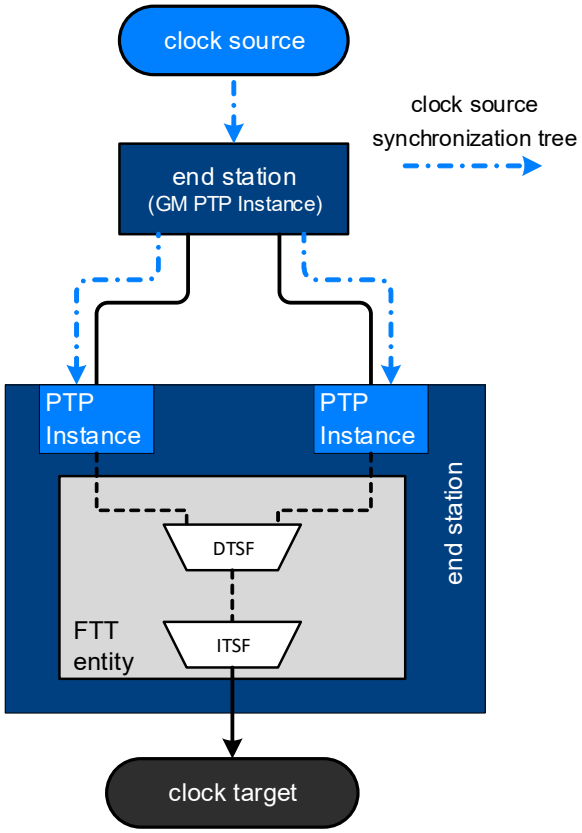


Figure J-3— FTT entity in example dual-homed network

Because there is just one GM in this network topology, the FTT entity supports increased time availability and time distribution integrity, but not GM integrity. Also, because there are just two time distribution paths, the limited integrity is lost if one of these two time distribution paths becomes faulty. Thus, the dual-homed network topology would be suitable for fail-stop applications in which the integrity of time distribution is important, but the integrity of the GM itself is not important.

J.2.3 Dual-star network

In dual redundant star network topologies, every end station is connected independently, in a star fashion, to a Bridge and to a redundant Bridge. A failure of any connection or of any Bridge is overcome by using the second connection or the second Bridge.

The example dual-star network shown in Figure J-4 provides two independent inputs, one from each GM through their associated PTP Instances, to each FTT entity. With just two independent inputs to each FTT

entity, this dual-star network topology can only detect a fault, but cannot overcome the fault, which makes it suitable for fail-stop applications (as opposed to fail-operational applications).

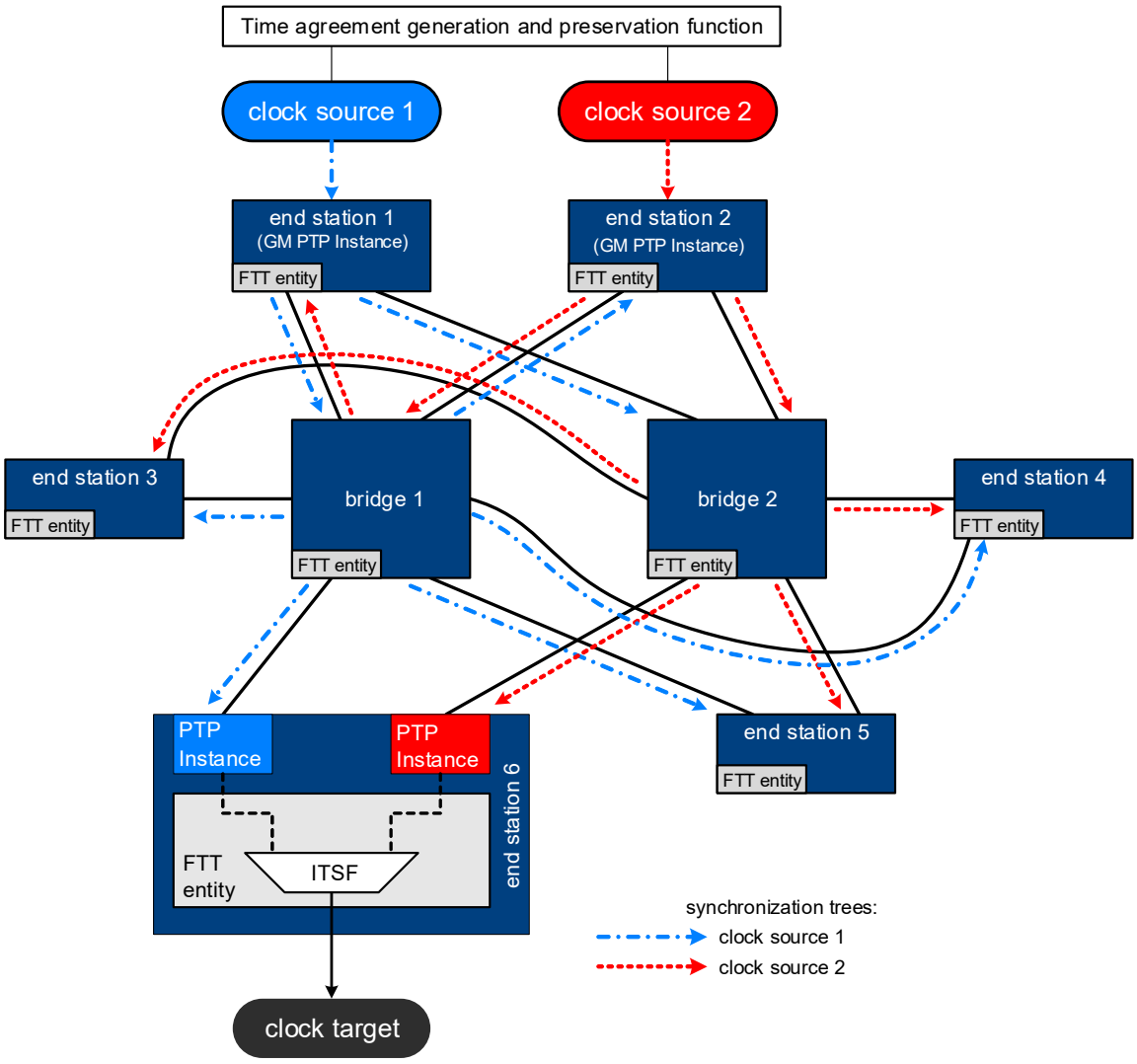


Figure J-4— FTT entity in example dual-star network with fail-stop time integrity

In contrast, the example dual-star network shown in Figure J-5 provides three independent inputs from each of the three GMs to the FTT entity of the two Bridges, two independent inputs from each of two GMs to the FTT entity of any end station, and two dependent inputs from a third GM to the FTT entity of end stations 3, 5, and 6. Even though the two dependent inputs originating from clock source 3's GM to the FTT entity of any end station have more than one dependency (i.e., the common clock source 3 and either bridge 1 or bridge 2), the FTT instance's DTSF is configured to only use the dependency on clock source 3 to determine the trustworthiness of these inputs for the following reasons:

a) Using clock source 3 as the dependent element results in the use of one DTSF instance, with two inputs, and generates three input times to the ITSF instance.
   — If the DTSF instance determines that the time values from the two inputs from clock source 3 match and, thus, can be trusted, then there is no need to pass clock source 3 through a DTSF instance with either clock source 1 or with clock source 2 as this is done in the ITSF.
   — If the DTSF instance determines that the two arriving times from clock source 3 do not match and, thus, cannot be trusted, then clock source 3 does not provide an applicable input to the ITSF

instance and is removed from contention as a trustworthy source of time to the ClockTarget entity. With this result, there is again no need to pass clock source 3 through a DTSF instance with either clock source 1 or with clock source 2.

— The result is three inputs to the ITSF, which enables trust determination when there is up to one fault.

b) Using bridge 1 and bridge 2 as the dependent element results in the use of two DTSF instances, each with two inputs. This results in just two inputs to the ITSF instance. Not finding trust in either DTSF instance results in an inability to determine trust at the ITSF instance. This configuration can detect a fault but cannot find trust when there is one fault in the system.

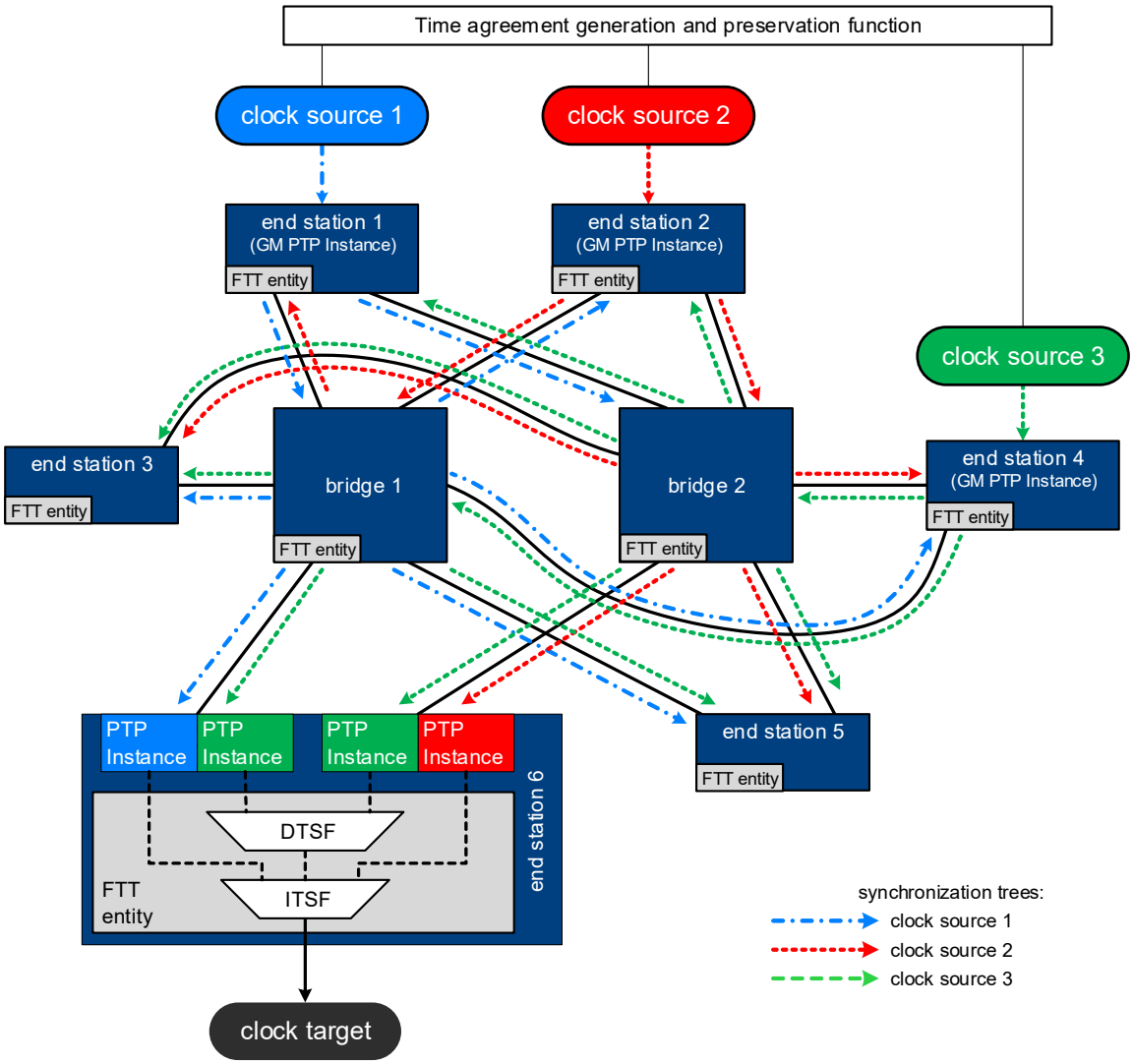


Figure J-5— FTT entity in example dual-star network with fail-operational time integrity

This dual-star network topology is suitable for fail-operational applications that can continue running when there is up to one fault in the timing function.

J.2.4 Ring network

A basic ring network topology provides a low fan-out network with availability recovery for a single fault. However, the following fundamental characteristics of the ring network topology impede the FTT entity's goals of maintaining time availability and integrity during fault conditions:

— The low fan-out characteristic of a ring can prevent an FTT entity from receiving all independent inputs and, thus, impedes its ability to achieve enhanced availability and integrity (see 20.1.3).
— The forwarding path of traffic, which includes PTP messages, is rearranged based on the location of a fault in the ring. This could lead to delay or loss of PTP messages, which in turn could affect a PTP timeReceiver's ability to retain a good PTP time.

The Ethernet ring protection mechanisms defined in ITU-T Recommendation G.8032/Y.1344 [B27a] use a mechanism called the ring protection link (RPL) to introduce a break in the ring by blocking frames. This break prevents the formation of loops in the ring, and it determines which direction around the ring a frame takes to get to its destination. When a failure introduces another break in the ring, the nodes on both sides of this break are reconfigured to block frames and the RPL is reconfigured so it no longer blocks frames, which opens a new path for frames to get around the ring.

The ring network shown in Figure J-6 provides two dependent inputs and one independent input to the FTT entities at every bridge in the ring. This is possible because the break in the ring (due to the RPL) does not require any Bridge to receive all three domains from the same ring direction. For simplicity, the end stations that connect to these Bridges and the fault-tolerance of their respective timing functions are not considered in this example.

The same ring network is shown in Figure J-7 with a repositioned break in the ring, resulting from a fault. With this repositioned break, this ring network can provide two dependent inputs and one independent input to the FTT entity in only four of the six Bridges. The other two Bridges are provided with three dependent inputs (they all share at least one common Bridge on the ring) to their FTT entity. As a result, these two Bridges cannot achieve time integrity (see 20.1.3).

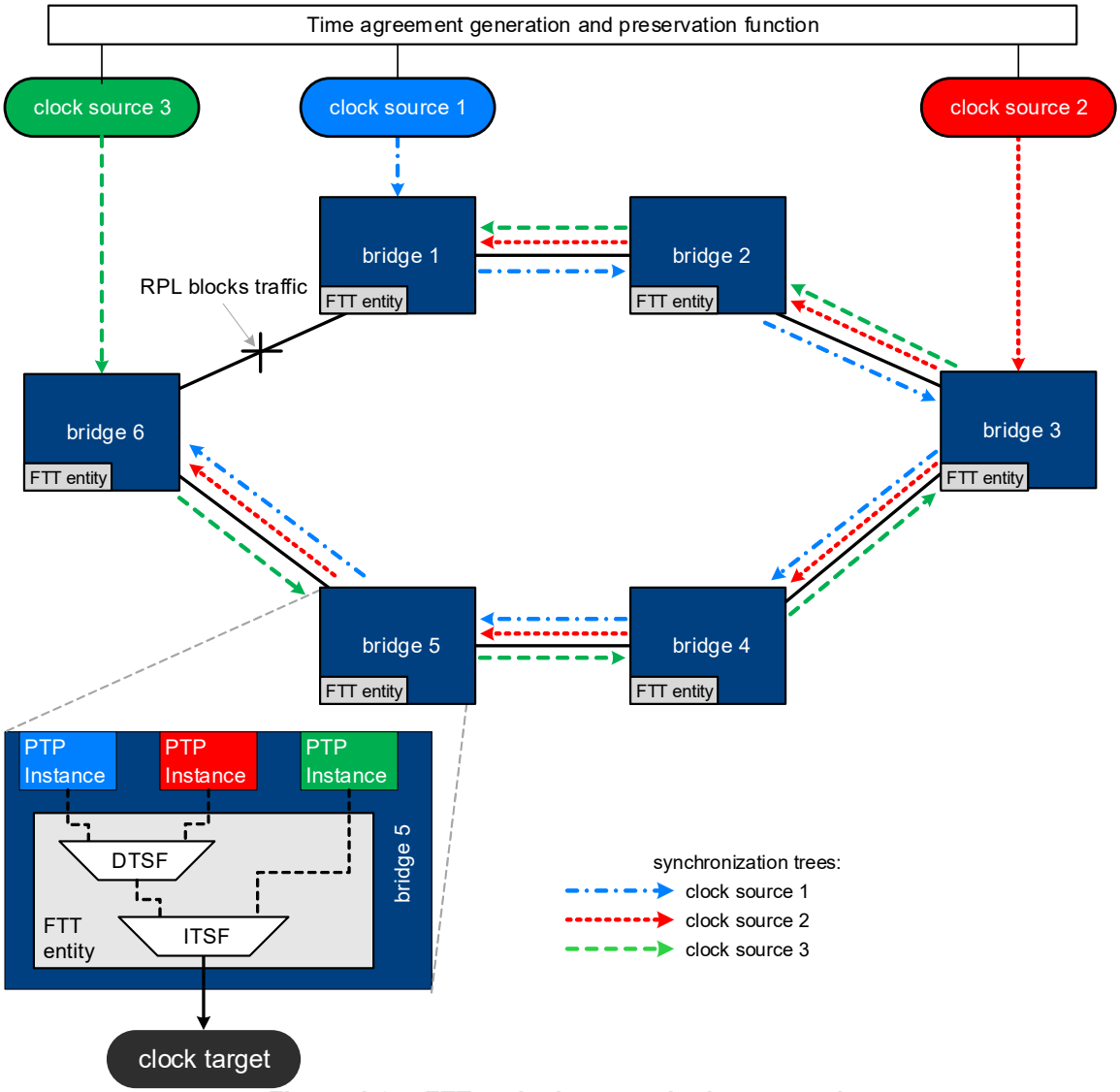


Figure J-6— FTT entity in example ring network

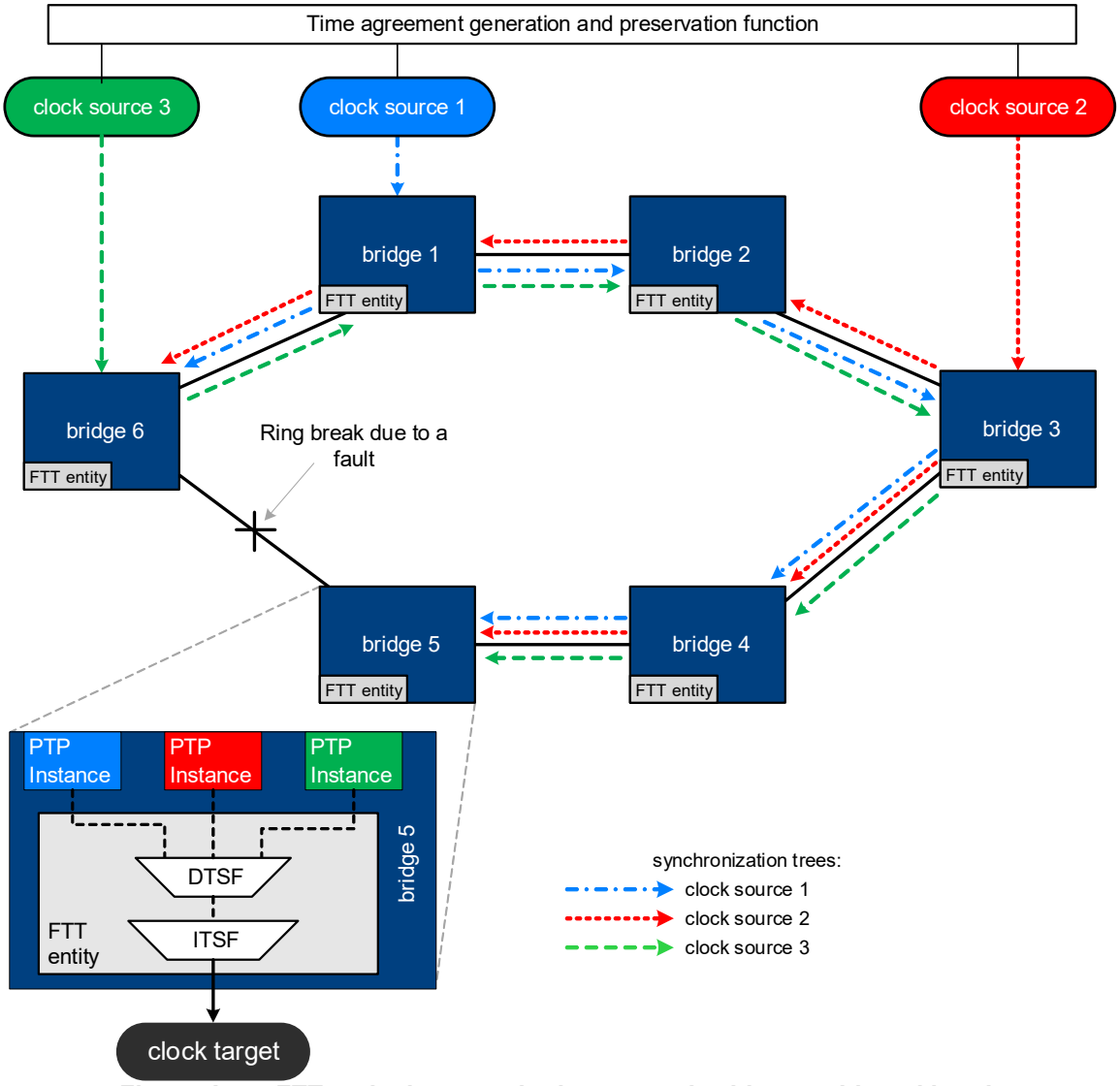


Figure J-7— FTT entity in example ring network with repositioned break

J.2.5 Mesh networks

A mesh network topology connects every network element directly to every other network element. The failure of one element does not adversely affect the ability of another working element to communicate with any other working element in the mesh.

Figure J-8 shows a mesh network of Bridges with three GMs. For simplicity, the end stations that connect to these bridges are not shown.

Because every Bridge has a direct connection to each GM, its FTT entity can receive an independent input from each of the three GMs (this is shown for only one Bridge in Figure J-8). Thus, this network topology would be suitable for fail-operational applications that can continue running when there is up to one fault in the time integrity function.

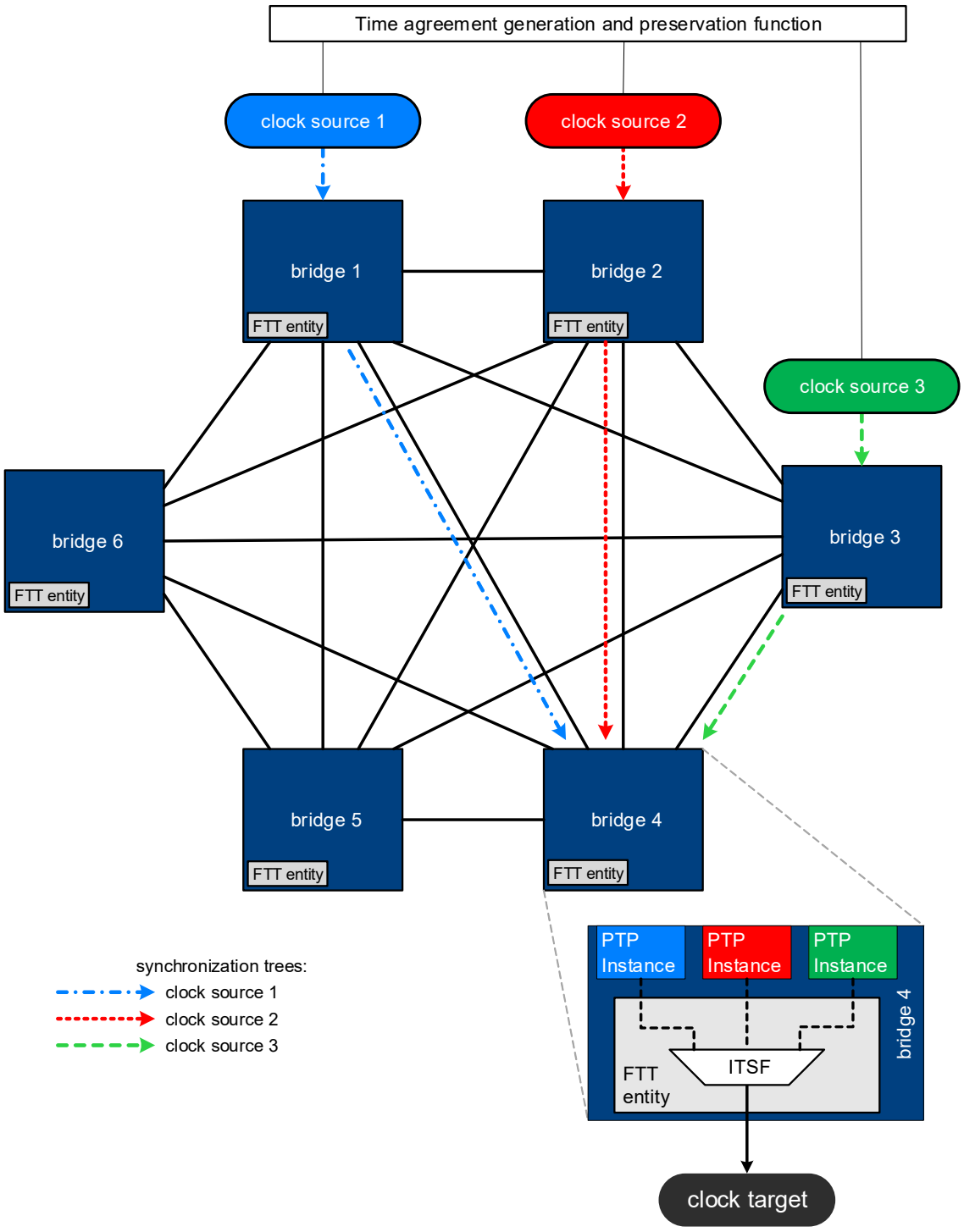


Figure J-8— FTT entity in example mesh network

Annex K

(informative)

Bibliography

Insert a new bibliography entry after entry [B3] as follows:

[B3a] Dolev, D., J. Halpern, H. R. Strong, "On the possibility and impossibility of achieving clock synchronization," 1984 (https://dl.acm.org/doi/abs/10.1145/800057.808720).

Insert a new bibliography entry after entry [B6] as follows:

[B6a] Hodson, R. F., W. Torres-Pomales, P. S. Miner, A. Loveless, "Failure-Tolerant Avionics for Crewed Space Systems Recommended Best Practices," 2024, (https://ntrs.nasa.gov/citations/20240009366).

Insert a new bibliography entry after entry [B10] as follows:

[B10a] IEEE Std 1588a™-2023, IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems—Amendment 3: Precision Time Protocol (PTP) Enhancements for Best Master Clock Algorithm (BMCA) Mechanisms.[10, 11]

Insert a new bibliography entry after entry [B27] as follows:

[B27a] ITU-T Recommendation G.8032/Y.1344, Ethernet ring protection switching.[12]

Insert a new bibliography entry after entry [B28] as follows:

[B28a] Lamport, L., P. M. Melliar-Smith, "Synchronizing clocks in the presence of Faults," 1985 (https://dl.acm.org/doi/10.1145/2455.2457).

Editor's note—The following three entries are not currently cited anywhere in the document. Citations should be added or these entries should be removed from the bibliography.

[B28b] Miner, P., A. Geser, L. Pike, J. Maddalon, "A Unified Fault-Tolerance Protocol," 2004 (https://ntrs.nasa.gov/citations/20040139869).

[B30a] Pease, M., R. Shostak, L. Lamport, "Reaching Agreement in the Presence of Faults," 1980 (https://lamport.azurewebsites.net/pubs/reaching.pdf).

[B32a] Ramanathan, P., K. G. Shin, R. W. Butler, "Fault-Tolerant Clock Synchronization in Distributed Systems," 1990 (https://ieeexplore.ieee.org/document/58235).

[10] IEEE publications are available from The Institute of Electrical and Electronics Engineers (https://standards.ieee.org).

[11] The IEEE standards or products referenced in Annex K are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.

[12] ITU-T publications are available from the International Telecommunications Union (https://www.itu.int/).